

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA  
INFORMÁTICA

MÁSTER UNIVERSITARIO EN INGENIERÍA DEL  
SOFTWARE E INTELIGENCIA ARTIFICIAL

**Estudio comparativo de los mecanismos de  
comunicación REST y gRPC en una arquitectura de  
microservicios**

**Comparative study of REST and gRPC  
communication mechanisms in a microservices  
architecture**

Realizado por  
**Manuel Díaz Alcalá**  
Tutorizado por  
**Lidia Fuentes Fernández**  
Departamento  
**Lenguajes y Ciencias de la Computación**

UNIVERSIDAD DE MÁLAGA  
MÁLAGA, SEPTIEMBRE DE 2025

## Resumen

La arquitectura de microservicios se ha consolidado como un paradigma clave para el desarrollo de sistemas escalables y eficientes. En este contexto, la elección del mecanismo de comunicación resulta decisiva para optimizar tanto el rendimiento como el consumo energético. Este trabajo compara los mecanismos de comunicación REST y gRPC en una arquitectura de microservicios, evaluando dos tipos de operaciones: la obtención y la creación de datos. Los experimentos se llevaron a cabo con volúmenes de datos pequeños, estándar y grandes, considerando métricas de rendimiento (tiempo de respuesta y *throughput*) y de consumo energético. Los resultados muestran que REST ofrece un mejor rendimiento al trabajar con datos pequeños y estándar, mientras que gRPC resulta más eficiente con grandes volúmenes. En términos energéticos, las diferencias son mínimas, salvo en escenarios con datos grandes, donde gRPC presenta una ligera ventaja. Además, se confirmó una relación entre el rendimiento y el consumo energético en la comunicación entre microservicios. En conjunto, este estudio aporta evidencia empírica que puede orientar a desarrolladores y arquitectos en la selección del mecanismo de comunicación más adecuado según el tipo de operación, el tamaño de los datos y las necesidades del sistema.

**Palabras clave:** mecanismo de comunicación, REST, gRPC, arquitectura de microservicios, *throughput*, tiempo de respuesta, consumo energético, obtención de datos, creación de datos.

### **Abstract**

Microservice architecture has become a key paradigm for developing scalable and efficient systems. In this context, the choice of communication mechanism is crucial to optimize both performance and energy consumption. This work compares the REST and gRPC communication mechanisms in a microservices architecture, evaluating two types of operations: data fetching and data creation. The experiments were conducted with small, standard, and large data volumes, considering performance metrics (response time and throughput) as well as energy consumption. The results show that REST provides better performance when working with small and standard data, while gRPC is more efficient with large volumes. In terms of energy consumption, the differences are minimal, except in scenarios with large data volumes, where gRPC presents a slight advantage. Furthermore, a correlation between performance and energy consumption in microservices communication was confirmed. Overall, this study provides empirical evidence that can guide developers and architects in selecting the most appropriate communication mechanism according to the type of operation, data size, and system requirements.

**Keywords:** communication mechanism, REST, gRPC, microservices architecture, throughput, response time, energy consumption, data fetching, data creation.

## Índice

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| 1     | Introducción.....                                                    | 6  |
| 1.1   | Motivación.....                                                      | 11 |
| 1.2   | Objetivos.....                                                       | 14 |
| 2     | Estado del arte .....                                                | 16 |
| 2.1   | Fundamentos técnicos de arquitectura de microservicios .....         | 16 |
| 2.2   | Fundamentos técnicos de REST y gRPC .....                            | 22 |
| 2.2.1 | El estilo arquitectónico REST .....                                  | 22 |
| 2.2.2 | El marco de trabajo gRPC .....                                       | 25 |
| 2.3   | Literatura científica relacionada.....                               | 27 |
| 2.3.1 | Revisión de la literatura.....                                       | 27 |
| 2.3.2 | Evaluación.....                                                      | 32 |
| 2.4   | Formulación de hipótesis .....                                       | 33 |
| 3     | Metodología de desarrollo del entorno experimental .....             | 34 |
| 3.1   | Diseño del ecosistema basado en microservicios.....                  | 35 |
| 3.1.1 | Ecosistema con mecanismo de comunicación REST .....                  | 36 |
| 3.1.2 | Ecosistema con mecanismo de comunicación gRPC .....                  | 40 |
| 3.2   | Desarrollo y ejecución del ecosistema basado en microservicios ..... | 46 |
| 3.2.1 | Desarrollo del ecosistema con mecanismo de comunicación REST .....   | 46 |
| 3.2.2 | Desarrollo del ecosistema con mecanismo de comunicación gRPC .....   | 46 |
| 3.2.3 | Entorno de ejecución .....                                           | 46 |
| 4     | Metodología experimental.....                                        | 48 |
| 4.1   | Elección de herramientas de medición.....                            | 48 |
| 4.2   | Definición del marco experimental.....                               | 48 |
| 4.2.1 | Parámetros de medida .....                                           | 48 |
| 4.2.2 | Escenarios .....                                                     | 48 |
| 4.3   | Ejecución de los experimentos.....                                   | 49 |
| 4.3.1 | Experimentos para peticiones de obtención de datos .....             | 49 |
| 4.3.2 | Experimentos para peticiones de creación de datos .....              | 51 |
| 5     | Análisis estadístico .....                                           | 54 |
| 5.1   | Experimentos de obtención de datos.....                              | 55 |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| 5.1.1 Experimentos de obtención de volúmenes de datos pequeños .....      | 55 |
| 5.1.2 Experimentos de obtención de volúmenes de datos estándar .....      | 60 |
| 5.1.3 Experimentos de obtención de volúmenes de datos grandes .....       | 65 |
| 5.2 Experimentos de creación de datos .....                               | 70 |
| 5.2.1 Experimentos de creación de volúmenes de datos pequeños .....       | 70 |
| 5.2.2 Experimentos de creación de volúmenes de datos estándar .....       | 75 |
| 5.2.3 Experimentos de creación de volúmenes de datos grandes .....        | 80 |
| 5.3 Correlación entre el tiempo de respuesta y el consumo energético..... | 85 |
| 6 Evaluación de los resultados.....                                       | 87 |
| 6.1 Respuestas a las preguntas de investigación .....                     | 87 |
| 6.2 Amenazas a la validez.....                                            | 89 |
| 6.2.1 Validez interna .....                                               | 89 |
| 6.2.2 Validez externa.....                                                | 89 |
| 6.2.3 Validez de constructo .....                                         | 89 |
| 6.2.4 Validez estadística-conclusiva.....                                 | 90 |
| 7 Conclusiones.....                                                       | 91 |
| 8 Líneas de investigación futuras .....                                   | 93 |
| Referencias.....                                                          | 95 |

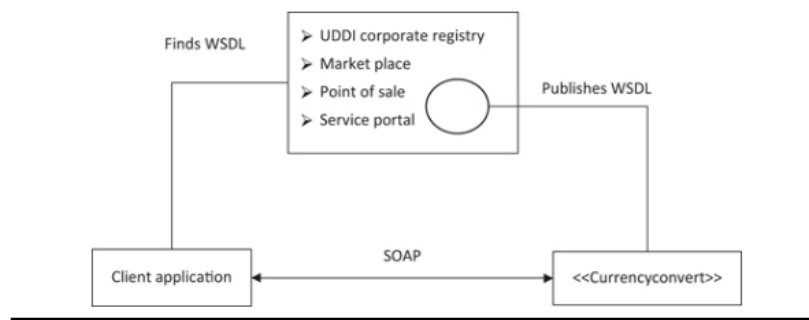
## 1 Introducción

En el contexto del desarrollo de software actual, la arquitectura de microservicios se ha consolidado como una respuesta efectiva a las limitaciones inherentes a los sistemas monolíticos y como una evolución pragmática de las arquitecturas orientadas a servicios (Service-Oriented Architecture, SOA). A diferencia del enfoque monolítico, en el que los distintos módulos de una aplicación dependen de recursos compartidos y no pueden ejecutarse de forma independiente, la arquitectura de microservicios propone una descomposición funcional del sistema en unidades cohesivas y autónomas, llamadas microservicios, que se comunican exclusivamente mediante el intercambio de mensajes. Esta separación permite abordar de manera efectiva problemas comunes en los sistemas monolíticos, como la dificultad para escalar, mantener o desplegar aplicaciones sin incurrir en costosos reinicios o bloqueos del sistema. Tal como detallan Dragoni et al. [1], los microservicios ofrecen ventajas significativas en términos de mantenibilidad, escalabilidad, independencia tecnológica y flexibilidad operativa. Además, su compatibilidad con prácticas modernas como la integración y entrega continua, así como su afinidad con herramientas de contenedorización, refuerzan su adopción en entornos empresariales dinámicos. Desde una perspectiva organizativa, los microservicios también facilitan una alineación entre los equipos de desarrollo y las capacidades de negocio, promoviendo modelos ágiles como el principio "*you build it, you run it*", popularizado en 2006 por Werner Vogels, entonces CTO de Amazon [2].

En una arquitectura de microservicios, existen diversos mecanismos de comunicación que permiten la interacción y el intercambio de datos entre los distintos servicios. Aunque en la práctica la mayoría de los desarrolladores tienden a utilizar REST (Representational State Transfer) debido a su simplicidad y amplia adopción, existen otros mecanismos igualmente relevantes que resultan interesantes de estudiar para comprender mejor las alternativas disponibles. Asimismo, es fundamental conocer la evolución histórica de los diferentes mecanismos de comunicación entre servicios web para comprender el contexto y las razones que han motivado la aparición de diversas alternativas.

Entre finales del siglo XX y comienzos del siglo XXI, en SOA, los servicios web utilizaban WSDL (Web Services Description Language) para describir interfaces, UDDI (Universal Description, Discovery, and Integration) para publicar y buscar esas interfaces y SOAP (Simple Object Access Protocol) para construir mensajes. WSDL es un estándar abierto basado en XML (Extensible Markup Language) que se utiliza para exponer la interfaz de un servicio web y consta de dos partes: una parte abstracta y una parte concreta. La parte abstracta describe las capacidades funcionales del servicio, es decir, describe las operaciones del servicio web. La parte concreta de la descripción de un servicio especifica la URL y el puerto donde el servicio está disponible. Con un documento WSDL, un servicio se describe utilizando seis elementos: <definitions>, <types>, <message>, <portType>, <binding> y <service>. <definitions> es el elemento raíz del WSDL, el cual contiene el resto de elementos. La parte abstracta de la descripción del servicio está formada por los elementos <types>, <message> y <portType>, mientras que la parte concreta la componen <binding> y <service>. WSDL incluye

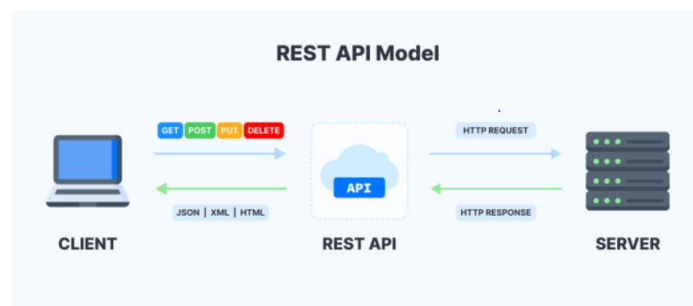
información sobre los tipos de datos que utiliza, los parámetros que requiere y devuelve, los agrupamientos de funcionalidades, el protocolo que debe utilizarse para acceder al servicio y la ubicación o dirección del servicio [3, pp. 13, 113-115]. Las descripciones de los servicios deben ser publicadas en un registro, de modo que los clientes puedan buscar y localizar los servicios que necesiten. Para publicar las interfaces y permitir su búsqueda, los servicios web definen un protocolo denominado UDDI. Se trata de un protocolo abierto basado en XML, utilizado para publicar los detalles de los servicios y sus descripciones WSDL, así como para buscar y localizar los WSDL necesarios [3, p. 13]. En 1998, Dave Winer, Don Box y Bob Atkinson, en colaboración con Microsoft desarrollaron SOAP y, debido a su rápida popularización, en 2003 fue formalmente recomendado por el W3C (World Wide Web Consortium) con la publicación de la especificación SOAP 1.2 [4]. SOAP es un protocolo de mensajería abierto basado en XML, mediante el cual el cliente invoca el método del servicio. Por su parte, XML es un protocolo basado en texto, y solo puede ser interpretado de la misma manera por todos los sistemas operativos, por lo que los servicios web adquieren de forma inherente la propiedad de independencia de plataforma. La relación entre WSDL, UDDI y SOAP se puede entender haciendo uso de la Figura 1. Como se puede ver en la imagen, *Currencyconvert* es el objeto servidor que publica su WSDL a través de cualquiera de los mecanismos, como el registro corporativo UDDI de la instalación del proveedor, mercado, punto de venta o portal de servicios. Un consumidor busca mediante cualquiera de estos métodos y encuentra el servicio que necesita. Después de obtener el WSDL del servicio requerido, el consumidor construye una solicitud SOAP según la semántica de llamada especificada en el WSDL. Finalmente, invoca la función requerida usando una llamada SOAP [3, p. 14].



**Fig. 1.** Relación entre WSDL, UDDI y SOAP [3, p. 14].

En 1999, se publicó el documento técnico RFC 2616 que definía el protocolo HTTP/1.1 [5]. Uno de los autores de ese documento, Roy Fielding, en el año 2000 formuló en su tesis doctoral [6] los principios del estilo arquitectónico para sistemas hipermedia distribuidos REST. Según Fielding, REST establece un conjunto de restricciones arquitectónicas que, cuando se aplican en conjunto, promueven la escalabilidad de las

interacciones entre componentes, la generalidad de las interfaces, el despliegue independiente de los componentes y el uso de componentes intermedios para reducir la latencia en las interacciones, reforzar la seguridad y encapsular sistemas heredados. Por su parte, una API RESTful (también conocida como API REST) es una implementación práctica de los principios del estilo arquitectónico REST en una interfaz de programación de aplicaciones (Application Programming Interface, API), que utiliza el protocolo de comunicación HTTP (Hypertext Transfer Protocol) para realizar operaciones sobre recursos [7, p. 42]. Como se puede observar en la Figura 2, en el modelo de API REST, el cliente realiza peticiones al servidor usando métodos estándar de HTTP (como GET, POST, PUT o DELETE), y los datos se intercambian generalmente en formato JSON (JavaScript Object Notation). La API actúa como una interfaz que permite al cliente interactuar con los recursos del servidor, gestionando las solicitudes y devolviendo respuestas que contienen la representación del recurso solicitado. eBay fue la primera empresa en explotar las APIs basadas en REST. Introdujo su API REST con socios seleccionados en noviembre del año 2000. Posteriormente, empresas como Amazon, Delicious y Flickr comenzaron a ofrecer APIs basadas en REST. Luego, Amazon Web Services (AWS) aprovechó el auge de la Web 2.0 y, en 2006, proporcionó APIs REST a los desarrolladores para facilitar el consumo de servicios en la nube de AWS. Más adelante, Facebook, Twitter, Google y otras empresas comenzaron a utilizar este enfoque [8, p. 5]. De esta forma, a principios de la segunda década del siglo XXI, REST ya se había consolidado como el estándar en el desarrollo de servicios web por encima de SOAP y, a día de hoy, es el estilo arquitectónico más usado en APIs.

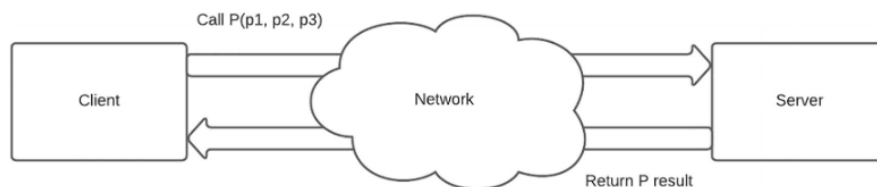


**Fig. 2.** Modelo de una API REST [9].

La llamada a procedimiento remoto (Remote Procedure Call, RPC) es un protocolo de red para establecer llamadas a procedimientos en una computadora remota utilizando un servidor de aplicaciones, también conocido como modelo cliente/servidor. La primera descripción de la llamada a procedimiento remoto data de 1976 en el estándar RFC 707 [10]. Con RPC, un cliente invoca una función o procedimiento del servidor con parámetros. Cuando el servidor recibe la solicitud, envía la respuesta requerida de vuelta al cliente. El cliente espera a que el servidor responda y no puede realizar otras operaciones hasta que el servidor haya terminado de procesar la solicitud. La Figura 3



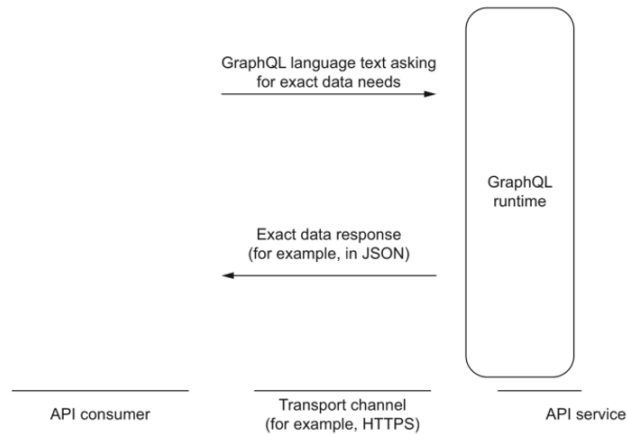
muestra el principio del modelo RPC [11, p. 86]. A diferencia de una API REST, en una API basada en RPC, las operaciones se ejecutan mediante funciones con parámetros, sin depender de verbos HTTP ni URLs específicas, pero sí que ambas tienen en común que hacen uso del protocolo HTTP. En los años 2000, Google empezó a trabajar en el proyecto gRPC (Google Remote Procedure Calls), un marco de trabajo RPC de alto rendimiento que puede ejecutarse en cualquier entorno, el cual la empresa americana hizo de código abierto en 2014. gRPC utiliza Protocol Buffers (*protobuf*) como lenguaje de descripción de interfaces (Interface Description Language, IDL), y la serialización y deserialización de mensajes se realiza en formato binario sobre una conexión HTTP/2 [11, pp. 86-87]. Google desarrolló gRPC con el objetivo de crear un sistema eficiente, escalable y multiplataforma para la comunicación remota entre servicios distribuidos, pensado especialmente para entornos de alto rendimiento. En los últimos años, gRPC ha ganado mucha popularidad como mecanismo de comunicación entre microservicios.



**Fig. 3.** Principios del modelo RPC [11, p. 86].

En 2011, Facebook enfrentaba desafíos para mejorar el rendimiento de su sitio web en navegadores móviles. Por ello, comenzaron a desarrollar su propia aplicación móvil utilizando tecnologías nativas. Sin embargo, las APIs no eran suficientes debido a la complejidad de datos jerárquicos y recursivos, y querían optimizar las llamadas de red. Hay que tener en cuenta que, por aquel entonces, la velocidad de la red móvil en algunas regiones del mundo era del orden de Kb/s. Contar con una aplicación móvil rápida y de alta calidad sería clave para su éxito, ya que sus usuarios ya habían empezado a migrar a dispositivos móviles. En 2012, los ingenieros de Facebook Lee Byron, Dan Schafer y Nick Schrock se unieron para crear GraphQL. Inicialmente, se utilizó para diseñar y desarrollar la función de noticias de Facebook, pero más adelante se aplicó en toda su infraestructura. Facebook lo utilizó internamente antes de ser liberado como código abierto en 2015. Fue entonces cuando publicaron la especificación de GraphQL y su implementación en JavaScript para el público. Poco después, comenzaron a surgir implementaciones en otros lenguajes de la especificación de GraphQL, incluyendo Java. GraphQL es un lenguaje declarativo para consultas y manipulaciones, además de un entorno de ejecución del lado del servidor para APIs [8, p. 458]. En el *backend*, una arquitectura basada en GraphQL necesita un entorno de ejecución. Ese entorno de ejecución proporciona una estructura para que los servidores describan los datos que se expondrán en sus APIs. Esta estructura es lo que en el mundo de GraphQL se conoce

como "esquema". Un consumidor de la API puede usar el lenguaje GraphQL para construir una solicitud en texto que represente exactamente los datos que necesita. El cliente envía esa solicitud al servicio de la API a través de un canal de transporte (por ejemplo, HTTPS). La capa de entorno de ejecución de GraphQL acepta la solicitud en texto, se comunica con otros servicios del *backend* para reunir una respuesta de datos adecuada y luego envía esos datos de vuelta al consumidor en un formato como JSON. La Figura 4 resume la dinámica de esta comunicación [12, pp. 4-5]. En contraste con las APIs REST, donde tienes que definir un *endpoint* para cada caso de uso, en las APIs basadas en GraphQL se expone un único *endpoint*, por lo que no necesitan cambios constantes como REST, mejorando así la velocidad de desarrollo y la capacidad de iteración [8, pp. 459-460]. En la actualidad, GraphQL es uno de los mecanismos de comunicación más utilizados, junto a REST y gRPC.



**Fig. 4.** GraphQL es un lenguaje y un entorno de ejecución [12, p. 5].

Además de los mecanismos de comunicación entre servicios web previamente mencionados, existen otras soluciones orientadas a la gestión de eventos y la transmisión de datos en tiempo real. Entre ellas destacan Webhooks, que permite la notificación unidireccional basada en eventos mediante solicitudes HTTP, y WebSocket, que habilita una comunicación bidireccional persistente entre cliente y servidor [13].

Habitualmente, la selección del mecanismo de comunicación se basa principalmente en criterios de rendimiento. No obstante, en la actualidad resulta clave también considerar el consumo energético. En [14] Freitag et al. examinaron la evidencia disponible sobre los impactos climáticos actuales y proyectados de las tecnologías de la información y la comunicación (TIC). Analizaron estudios revisados por pares, estimando que la participación de las TIC en 2020 en las emisiones globales de gases de efecto invernadero (principalmente CO<sub>2</sub>) se situaba entre el 1,8% y el 2,8% del total global. Sin embargo, los hallazgos de los autores indicaban que todas las estimaciones publicadas

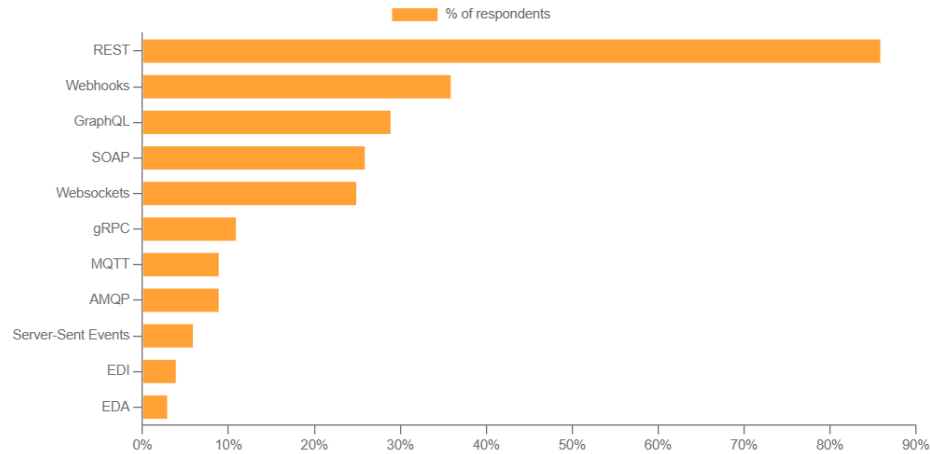
subestimaban la huella de carbono de las TIC, posiblemente hasta en un 25 %, al no considerar completamente todas las cadenas de suministro y el ciclo de vida completo de las TIC. Bajo su punto de vista, la participación real de las TIC en las emisiones podría estar entre el 2,1% y el 3,9%. Así pues, reducir la huella de carbono en las TIC es de vital importancia para cumplir con el objetivo de la Unión Europea de conseguir en 2030, al menos, un 55% menos de emisiones netas de gases de efecto invernadero (emisiones una vez deducidas las absorciones) en comparación con 1990 [15]. Por tanto, es fundamental que los desarrolladores cuenten con herramientas y criterios que les permitan seleccionar el mecanismo de comunicación más adecuado. Más allá del rendimiento y de otros atributos de calidad (como la flexibilidad, el mantenimiento, la escalabilidad...), el consumo energético asociado a las comunicaciones debe incorporarse como un factor clave en la toma de decisiones, tanto desde una perspectiva técnica como ecológica.

## 1.1 Motivación

Durante años, REST ha sido el estándar de facto para la construcción de APIs, gracias a su simplicidad, amplio soporte y compatibilidad con el protocolo HTTP. REST permite una comunicación entre microservicios web fácilmente accesible desde lenguajes de programación y herramientas tradicionales, lo que lo convierte en una opción natural para incorporar una funcionalidad a una aplicación web. Anualmente, Postman elabora el State of the API Report, un informe que ofrece una visión integral sobre el estado y las tendencias de las APIs a nivel global, basado en encuestas a miles de desarrolladores y profesionales del sector. Este informe se ha consolidado como una de las fuentes más relevantes para entender la evolución, adopción y desafíos del ecosistema de APIs.

Aunque el informe de 2024 de Postman [16] no incluye datos sobre el porcentaje de encuestados que usaron cada uno de los estilos arquitectónicos de APIs, dicha información sí fue proporcionada en ediciones anteriores. Según el informe de 2023 [17], REST fue, con diferencia, la arquitectura de API más utilizada, aunque perdió ligeramente terreno frente a nuevas alternativas. Ese año, el 86% de los encuestados afirmó utilizar REST, frente al 89% en 2022 y al 92% en 2021. Por su parte, SOAP registró un descenso notable: en 2023 fue utilizado por solo el 26% de los encuestados, comparado con el 34% del año anterior, lo que lo relegó al cuarto puesto en uso, después de haber ocupado el tercero en 2022. En lo que respecta a GraphQL, en 2023 este ocupó el tercer puesto, superando a SOAP y siendo utilizado por el 29% de los encuestados. Asimismo, gRPC ha experimentado un crecimiento sostenido en los últimos años. En 2020, solo el 6,9% de los encuestados indicó utilizar gRPC [18], mientras que en 2023 esta cifra aumentó al 11%.

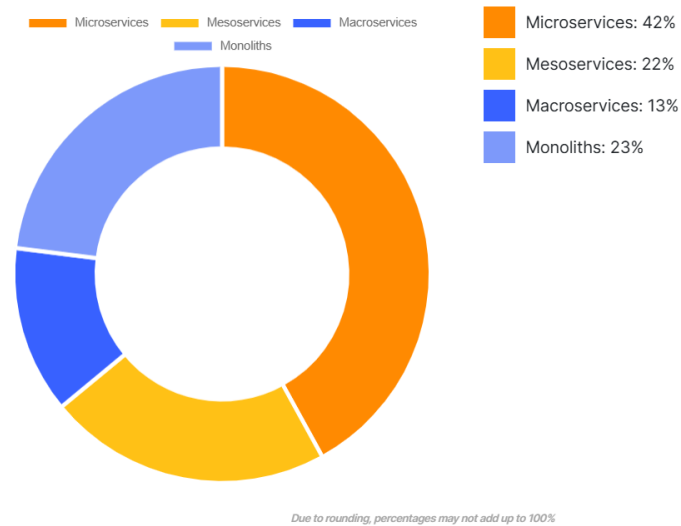
En la Figura 5, se puede ver una clasificación de la utilización de los diferentes estilos de APIs en 2023, según las respuestas de los encuestados.



**Fig. 5.** Uso de estilos de APIs [17].

Atendiendo a la imagen anterior, observamos que Webhooks y WebSocket son muy usados, sin embargo, al tratarse de tecnologías basadas en eventos y transmisión de datos, han sido excluidos de este estudio. Por su parte, SOAP no ha sido considerado, dada su popularidad decreciente en los últimos años. Obviamente, REST tenía que ser incluido en el estudio, ya que es el estilo más utilizado. Finalmente, entre GraphQL y gRPC se podría haber seleccionado cualquiera de los dos, pero se ha escogido gRPC porque se intuye que puede tener un gran crecimiento en los próximos años.

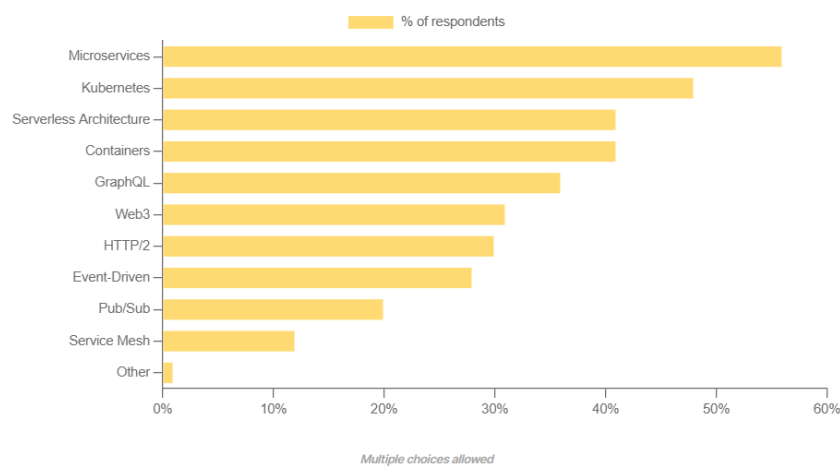
Existen varias alternativas de patrones de arquitectura de software para diseñar aplicaciones web, especialmente si tenemos en cuenta la granularidad de los servicios que componen la aplicación. Una aplicación web monolítica es aquella que no descompone la funcionalidad de la aplicación web en varios servicios. Si realizamos la descomposición en servicios, según su granularidad se clasifican en macroservicios, mesoservicios y microservicios. Por un lado, los macroservicios tienen una granularidad gruesa y gestionan múltiples responsabilidades o subdominios. Por otro lado, los mesoservicios tienen una granularidad media y maneja un solo dominio, pero agrupa varias funciones relacionadas. Finalmente, los microservicios cuentan con una granularidad fina y una única responsabilidad. La Figura 6 muestra qué porcentaje de encuestados usó cada patrón en 2023.



**Fig. 6.** Uso de patrones de arquitectura de software [17].

La arquitectura de microservicios fue la más empleada, con un 42 % de los encuestados que indicaron utilizarla. Muy por detrás se situó la arquitectura monolítica, con un 23 % de adopción.

Además, en 2022, en el informe de Postman [19] se preguntó a los participantes en las encuestas cuáles creían que serían las tecnologías del futuro, siendo la arquitectura de microservicios la ganadora, tal y como se puede ver en la Figura 7.



**Fig. 7.** Tecnologías de futuro [19].

En la actualidad, la arquitectura de microservicios es el patrón arquitectónico más empleado en el desarrollo de APIs y los informes analizados indican que esta tendencia continuará en los próximos años, por lo que se ha escogido la arquitectura de microservicios como patrón arquitectónico. Así pues, en este trabajo se realizará un estudio empírico donde se compararán los estilos arquitectónicos de APIs REST y gRPC en una arquitectura de microservicios en términos de rendimiento y consumo energético.

La motivación principal de este trabajo surge de la falta de estudios concluyentes y específicos que comparen REST y gRPC en el contexto de una arquitectura de microservicios. Aunque existen investigaciones previas centradas en el rendimiento y el consumo de energía [20], [21], [22], [23], [24], [25], [26], [27] y [28], los resultados no son del todo esclarecedores ni generalizables. En cuanto al consumo energético, ha sido abordado de forma muy limitada en la literatura científica, y no se han identificado estudios que lo analicen específicamente en entornos basados en arquitectura de microservicios. Además, solo en [21] se aplican técnicas de estadística descriptiva e inferencial para analizar los resultados, mientras que el resto de los estudios se limita únicamente al uso de estadística descriptiva. Por ello, se ha considerado especialmente relevante llevar a cabo una evaluación comparativa entre REST y gRPC, centrada tanto en el rendimiento como en el consumo energético, en un entorno actual y representativo, empleando para el análisis de los resultados técnicas de estadística descriptiva e inferencial.

## 1.2 Objetivos

Ante la creciente importancia de las aplicaciones basadas en microservicios y la falta de estudios detallados que realicen una comparación exhaustiva entre REST y gRPC, este trabajo busca proporcionar una valiosa aportación tanto a la comunidad académica como a los profesionales en el campo del desarrollo y la Arquitectura del Software. El objetivo principal de este estudio es llevar a cabo un análisis bien fundamentado, que permita tomar decisiones informadas sobre la elección del mecanismo de comunicación más adecuado en arquitecturas basadas en microservicios.

Asimismo, este estudio pretende proporcionar una guía práctica y de referencia para evaluar de manera comparativa REST y gRPC en términos de rendimiento y eficiencia energética, dos factores clave a tener en cuenta durante el desarrollo de los sistemas distribuidos actuales. Se espera que los resultados obtenidos establezcan una base robusta para investigaciones futuras que busquen profundizar en este ámbito, ampliando los criterios de comparación o incorporando nuevas tecnologías emergentes de composición de microservicios. Para lograr estos objetivos, se llevará a cabo un análisis teórico y empírico.

El análisis teórico se centrará en examinar los fundamentos técnicos de la arquitectura de microservicios, así como de los mecanismos de comunicación REST y gRPC. Además, se realizará un estudio comparativo de ambas tecnologías basado en una revisión previa de la literatura científica.

En cuanto al análisis empírico, en primer lugar, se desarrollarán dos entornos experimentales basados en arquitectura de microservicios: uno con mecanismo de comunicación REST y otro con gRPC. Cada entorno experimental implementará las mismas aplicaciones basadas en microservicios que serán ejecutadas en contenedores Docker, con el objetivo de generar un entorno experimental lo más realista posible. Se incluirán microservicios de creación y obtención de datos y también de autenticación. A su vez, los microservicios con operaciones de creación y obtención de datos estarán conectados a una base de datos. Para llevar a cabo el estudio, basándonos en la revisión de la literatura realizada previamente, se definirán las hipótesis relacionadas con las siguientes preguntas de investigación (o *research questions*, RQ) que guiarán el diseño de los experimentos:

- RQ1: ¿Qué mecanismo de comunicación tiene mejor rendimiento en la obtención de volúmenes de datos de diferente tamaño?
- RQ2: ¿Qué mecanismo de comunicación consume menos energía en la obtención de volúmenes de datos de diferente tamaño?
- RQ3: ¿Qué mecanismo de comunicación ofrece mejor rendimiento en la creación de volúmenes de datos de diferente tamaño?
- RQ4: ¿Qué mecanismo de comunicación consume menos energía en la creación de volúmenes de datos de diferente tamaño?
- RQ5: ¿Existe relación entre el rendimiento y el consumo de energía en la comunicación entre microservicios?

A partir de estas preguntas se diseñarán y ejecutarán las pruebas controladas que permitirán comparar el comportamiento de REST y gRPC en diferentes escenarios. Finalmente, se analizarán los resultados obtenidos que nos permitirán responder a cada una de estas preguntas.

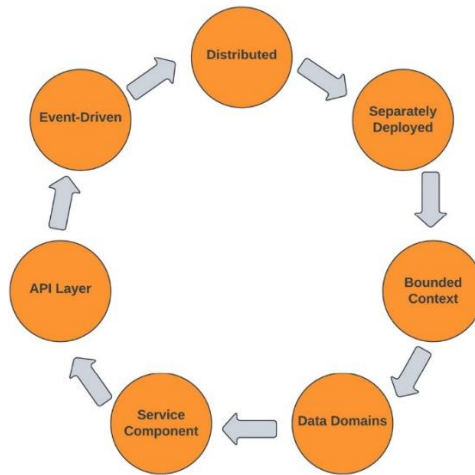
## 2 Estado del arte

### 2.1 Fundamentos técnicos de arquitectura de microservicios

Para definir el concepto de arquitectura de microservicios, previamente debemos conocer los conceptos de arquitectura software de un sistema, patrón arquitectónico y patrón de diseño. Existen muchas definiciones de arquitectura software de un sistema, pero una de las más reconocidas es la propuesta por Bass et al.: “La arquitectura software de un sistema es el conjunto de estructuras necesarias para razonar sobre el sistema, que comprende los elementos de software, las relaciones entre ellos y las propiedades de ambos” [29, p. 2]. Una estructura es simplemente un conjunto de elementos (módulos o componentes) unidos por una relación. Los sistemas de software están compuestos por múltiples estructuras, y ninguna de ellas por sí sola constituye la arquitectura completa. Un sistema de software comprende una cantidad infinita de estructuras, pero solo se pueden considerar arquitectónicas aquellas que permiten razonar sobre el sistema y sus propiedades. Por tanto, la arquitectura software de un sistema es, ante todo, una abstracción del sistema que selecciona ciertos detalles y suprime otros [29, pp. 2-3]. En algunos casos, los elementos arquitectónicos se combinan formando estructuras que resuelven problemas específicos. Estas estructuras, conocidas como “patrones arquitectónicos”, han demostrado ser útiles a lo largo del tiempo y en diversos dominios, por lo que han sido documentadas y difundidas. Ofrecen estrategias empaquetadas para resolver algunos de los problemas comunes que enfrenta un sistema [29, p. 18]. Frecuentemente, el término “patrón arquitectónico” es confundido con el concepto de “patrón de diseño”, ya que ambos ofrecen soluciones reutilizables a problemas comunes. Sin embargo, el alcance de un patrón arquitectónico es más amplio que el de un patrón de diseño. Mientras que los patrones arquitectónicos definen la estructura general y la organización de un sistema, los patrones de diseño se centran en soluciones específicas dentro de la arquitectura [30, pp. 3-6]. Existen diferentes tipos de patrones arquitectónicos, entre los que se encuentra el patrón de arquitectura de microservicios.

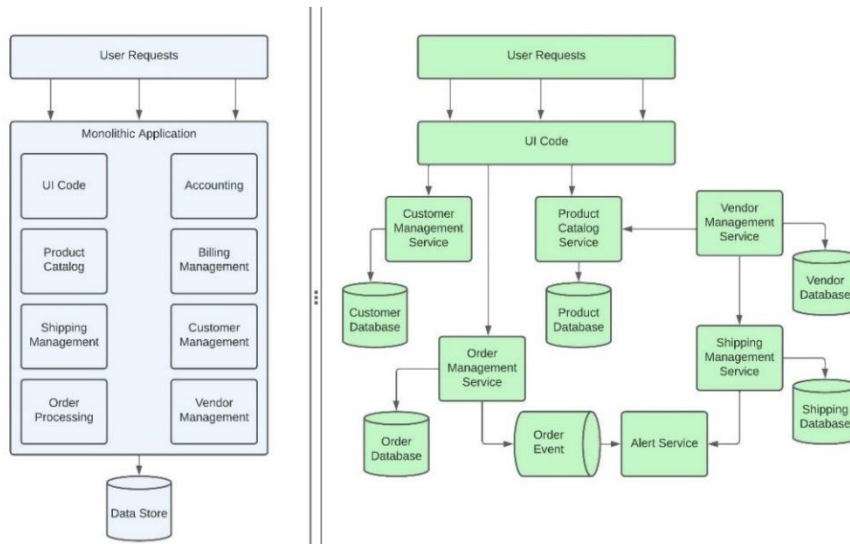
El patrón de arquitectura de microservicios promueve el uso de servicios pequeños en un contexto delimitado. La arquitectura de microservicios es una arquitectura distribuida, en la que los servicios son de grano fino y sirven para un propósito específico. En el patrón de arquitectura de microservicios, una aplicación se divide en componentes más pequeños y autónomos, conocidos como servicios. Cada servicio se ejecuta en un proceso y se conecta a otros servicios de forma síncrona o asíncrona. Cada microservicio es responsable de llevar a cabo una determinada función empresarial dentro de un contexto delimitado, y la aplicación completa es una colección de estos servicios débilmente acoplados. Los microservicios proporcionan una interoperabilidad consciente de los protocolos, por lo que quien llama al servicio necesita conocer el contrato y el protocolo del servicio al que llama [30, p. 22]. El diagrama de la Figura 8 explica las características de un patrón de arquitectura de microservicios.





**Fig. 8.** Características de la arquitectura de microservicios [30, p. 23].

En los últimos años, las arquitecturas de microservicios han sido la principal alternativa a las arquitecturas monolíticas. En contraste con las aplicaciones basadas en microservicios, una aplicación es monolítica cuando no solo se encarga de una tarea específica, sino que también lleva a cabo todos los pasos necesarios para completar una función de negocio. Por lo general, en este tipo de aplicaciones, la interfaz de usuario y el código necesario para acceder a los datos están empaquetados conjuntamente y la aplicación funciona como una unidad autosuficiente [30, p. 30]. En la Figura 9 se puede observar la diferencia entre una aplicación monolítica y una aplicación basada en microservicios.



**Fig. 9.** Arquitectura de aplicación monolítica frente a microservicios [30, p. 33].

Cada programador puede llevar a cabo la implementación del software de manera diferente, teniendo distintos enfoques y puntos de vista para resolver una necesidad o problema planteado. Sin embargo, es fundamental estandarizar el diseño y la construcción del software, con el objetivo de que la aplicación construida sea escalable, fácil de probar y favorezca la reutilización de componentes. Para ello, en el diseño y construcción del software es necesario el uso de patrones. Es esencial e importante, desde el inicio del desarrollo, establecer patrones de diseño adecuados. A continuación, se explican algunos de los patrones más importantes utilizados en el diseño de microservicios, especialmente al migrar de una aplicación monolítica hacia una arquitectura de microservicios, tal como se describen en [31].

- Domain-Driven Design

Domain-Driven Design (DDD) es un conjunto de herramientas que ayuda a diseñar e implementar software de calidad y alto valor, tanto en la parte estratégica como en la táctica, facilitando el diseño del software y su integración con el negocio. El patrón de diseño orientado al dominio se utiliza para desarrollar una visión del sistema a construir, generando un modelo del dominio del negocio desde el cual se extrae el conocimiento de las funcionalidades empresariales. Esto permite, por ejemplo, crear “casos inteligentes”, de manera que se pueda construir un lenguaje común entre desarrolladores, arquitectos y expertos en el dominio. La generación de diagramas aporta beneficios como: identificar subdominios, identificar funcionalidades que están estrechamente relacionadas con otras, identificar funcionalidades que son prioritarias o importantes para la empresa, dependencias entre funcionalidades, llamadas a otros sistemas externos o servicios de terceros, todo lo cual debe estar incluido en el modelo de dominio. Al descomponer un sistema en muchos microservicios pequeños que realizan una única funcionalidad de negocio, con el apoyo de DDD para delimitar los límites que cada microservicio debe tener, es posible obtener microservicios con alta cohesión.

- Service Discovery Pattern

Cuando se diseñan y crean microservicios, generalmente se utiliza una API, que ofrece los servicios necesarios para que la aplicación pueda interactuar con el *backend*. Los microservicios necesitan saber dónde están alojados esos servicios. Con el patrón Service Discovery, la idea es que los propios servicios, al iniciarse, se registren en una entidad llamada Service Registry, la cual mantiene el control de todos los servicios activos. De este modo, cuando se desea consumir un servicio, se consultan las instancias disponibles en el Service Registry. Cada servicio, al arrancar, indica su nombre y la dirección donde está ubicado, lo que permite que el Service Registry tenga un registro actualizado de todos los servicios disponibles. Además del registro inicial, los servicios deben enviar periódicamente señales de vida (conocidas como *heartbeats*) para que el Service Registry sepa que siguen disponibles. Si un servicio deja de enviar estas señales, el registro asumirá que ese servicio ya no está disponible y redirigirá todas las solicitudes a otras instancias disponibles. Este patrón permite ejecutar un servicio

específico sin conocer su dirección física. Para ello, se utiliza un balanceador de carga, que se apoya en el Service Registry para conocer la ubicación real del servicio. En la Figura 10 se muestra un ejemplo de este patrón.

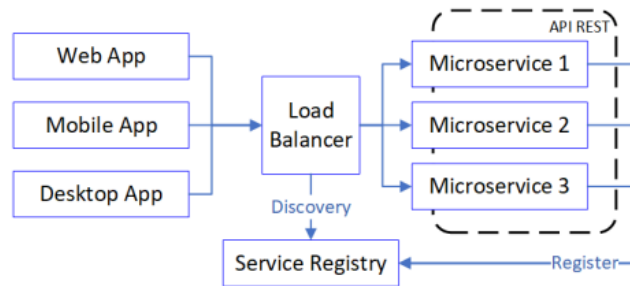


Fig. 10. Patrón Service Discovery [31].

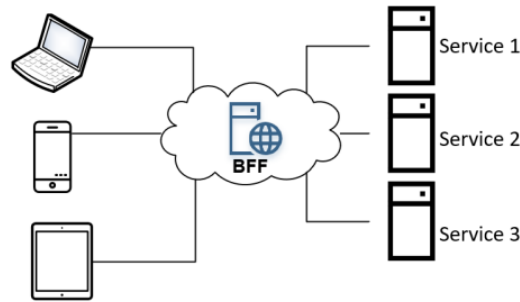
- Data-Driven Design

Los datos constituyen uno de los activos más críticos para cualquier organización; por lo tanto, ser capaz de recopilarlos y gestionarlos correctamente ofrece una ventaja competitiva frente a la competencia. Data-Driven Design (D-DD) consiste en realizar diseños a partir de los datos obtenidos mediante la analítica de una aplicación, lo que permite identificar información importante o relevante sobre los usuarios que la utilizan, replantear el diseño visual para facilitar la usabilidad y accesibilidad de la aplicación, así como reorganizar y adaptar el contenido según las necesidades del usuario, permitiendo, por ejemplo, maximizar las ventas. Si una aplicación ha sido desarrollada utilizando D-DD, existe una mayor transparencia al migrar a microservicios, ya que las funcionalidades del negocio ya se encuentran separadas, lo que da lugar a un acoplamiento débil. Los microservicios se inspiran en los principios propuestos por Domain-Driven Design, en el cual primero se debe definir un contexto delimitado para un problema de negocio específico, lo que se ajusta perfectamente al principio de responsabilidad única, utilizado en la creación de microservicios.

- Backend for Frontend Pattern

El patrón Backend for Frontend (BFF) es un patrón de arquitectura de software que mejora la forma en que se obtiene la información entre los clientes y los servidores alojados localmente o en la nube. El objetivo de este patrón es desacoplar las aplicaciones *frontend* de la arquitectura *backend*. Al utilizar este patrón, si se agrega un nuevo servicio, no es necesario realizar cambios entre las aplicaciones *frontend* y *backend*, sino simplemente adaptar las llamadas al servicio BFF. Su principal beneficio es permitir aislar el *backend* del *frontend*. Una ventaja adicional de este patrón es la reutilización de código, ya que todos los clientes pueden obtener datos desde el mismo

servidor BFF, lo que reduce la complejidad general del sistema. Un ejemplo de este patrón de arquitectura BFF se puede ver en la Figura 11.



**Fig. 11.** Ejemplo de patrón BFF [31].

- Adapter Microservices Pattern

El patrón Microservices Adapter adapta, según sea necesario, una API orientada al negocio construida mediante técnicas de mensajería ligera o RESTful con una API heredada o un servicio tradicional basado en SOAP. Esta adaptación es necesaria, por ejemplo, cuando un equipo de desarrollo no tiene control descentralizado sobre la fuente de datos de una aplicación. Un microservicio adaptador encapsula y traduce los servicios existentes en una interfaz REST basada en entidades. Este tipo de microservicio trata cada nueva interfaz de entidad como un microservicio independiente, y la construye, gestiona y escala de forma autónoma.

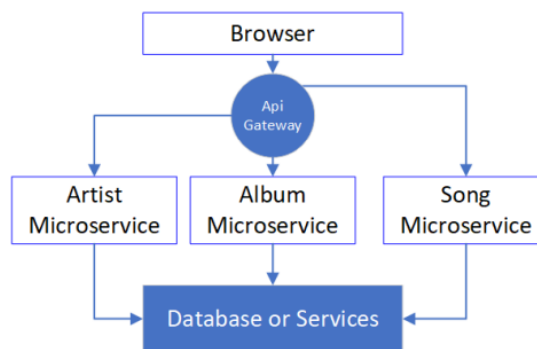
- Strangler Application Pattern

El patrón Strangler ayuda a gestionar el proceso de refactorización de una aplicación monolítica de forma progresiva. La idea consiste en aprovechar la estructura de las aplicaciones web, que están compuestas por URIs individuales asignadas a distintas funcionalidades del dominio de negocio, para dividir la aplicación en dominios funcionales y reemplazar cada uno de ellos, uno a uno, con una nueva implementación basada en microservicios. De este modo, la aplicación original y la nueva implementación coexisten dentro del mismo espacio de URIs durante un tiempo. A medida que se reemplazan más partes del sistema, el nuevo software refactorizado va sustituyendo al monolito hasta que este puede ser completamente retirado. El proceso implica crear una nueva versión del sistema en paralelo e ir transformándola de forma incremental, permitiendo que ambas versiones coexistan temporalmente mientras se redirige el tráfico de manera controlada hacia la nueva. Finalmente, se eliminan las partes antiguas y se consolida el sistema moderno. Este patrón favorece una adopción gradual de microservicios, reduce el riesgo asociado a grandes reescrituras y permite revertir el enfoque si surgen

problemas. Puede aplicarse tanto refactorizando el *backend* para alinearlo con una arquitectura de microservicios como modificando el *frontend* para adaptarse a dicha arquitectura y permitir la incorporación de nuevas funcionalidades.

- Shared Data Microservice Design Pattern

Este patrón indica que, en un proceso de migración desde una aplicación heredada hacia microservicios, durante la fase de transición de dicha migración, se puede considerar el uso temporal de un mismo repositorio de base de datos con el fin de que el proceso sea más fluido. Para ello, los límites de la funcionalidad de negocio deben haber sido claramente definidos con anterioridad, de modo que se creen microservicios con bajo acoplamiento y alta cohesión, apoyándose en los principios de DDD. Sin embargo, debe tenerse en cuenta que, si al finalizar el proceso de migración se mantiene un único repositorio de base de datos compartido, esta práctica puede considerarse un antipatrón. En la Figura 12 se puede ver un ejemplo de este patrón.

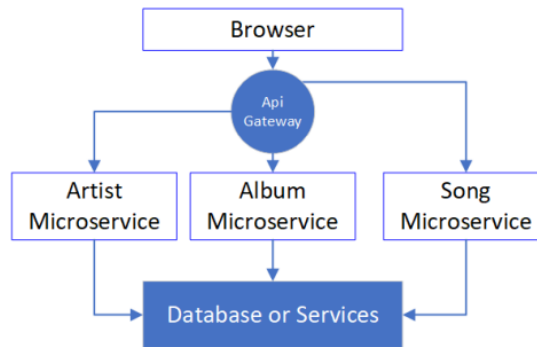


**Fig. 12.** Ejemplo de Shared Data Microservice Design Pattern [31].

- Aggregator Microservice Design Pattern

Una entidad es un objeto que se distingue principalmente por su identidad. Las entidades son los objetos en el proceso de modelado que tienen identificadores únicos. Los objetos entidad necesitan ciclos de vida bien definidos y una buena definición de cuál es la relación raíz de identidad. Un buen modelo entidad-relación tiene entidades bien definidas y cada entidad tiene un identificador específico y reconocido, aunque puede que nunca existan de forma independiente. Una combinación de entidades es un agregado. En los casos en que existe un conjunto de entidades que deben mantenerse consistentes, estas se denominan entidades dependientes. Es fundamental identificar la raíz del agregado, dado que esta determina el ciclo de vida de las entidades dependientes. Para los equipos de desarrollo, los patrones de entidad y agregado son útiles para identificar conceptos específicos del negocio que se mapean directamente en

microservicios, ejecutando funciones de negocio de extremo a extremo. La Figura 13 muestra un ejemplo de este patrón.



**Fig. 13.** Ejemplo de Aggregator Microservice Design Pattern [31].

## 2.2 Fundamentos técnicos de REST y gRPC

### 2.2.1 El estilo arquitectónico REST

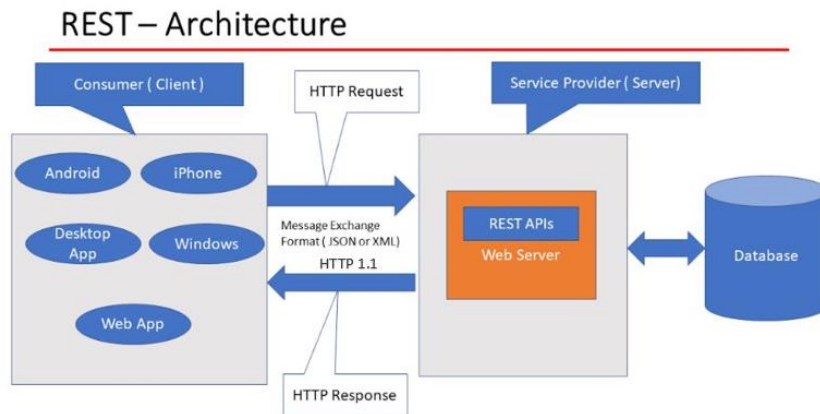
El estilo arquitectónico REST se basa en los siguientes principios:

- Todo es un recurso: cada cosa o información que se puede identificar y manipular en el sistema se considera un recurso [32, p. 7].
- Cada recurso es identificable por un identificador único (URI): dado que Internet contiene una gran variedad de recursos, todos ellos deben ser accesibles mediante URIs y deben ser identificados de forma única. Además, las URIs pueden estar en un formato legible para humanos, a pesar de que sus consumidores probablemente sean programas de software más que personas comunes [32, p. 8].
- Uso de los métodos HTTP estándar: el protocolo HTTP 1.1, definido en RFC 2616 y adoptado por REST como protocolo de comunicación, contempla ocho acciones, también conocidas como verbos HTTP (POST, GET, PUT, DELETE, HEAD, OPTIONS, TRACE, CONNECT). Los verbos POST, GET, PUT y DELETE son los más utilizados en el contexto de los recursos [32, pp. 8-10]. El método POST realiza una operación de creación enviando datos a un servidor; el método GET recupera datos de un recurso especificado; el método PUT realiza una operación de actualización enviando datos a un servidor; y el método DELETE realiza la operación de borrado [7, p. 43].
- Los recursos pueden tener múltiples representaciones: una característica clave de un recurso es que puede representarse de forma distinta a la que está almacenado. Así, puede solicitarse o publicarse en diferentes representaciones.

Siempre que el formato especificado sea compatible, el *endpoint* habilitado para REST deberá utilizarlo [32, pp. 10-11].

- Comunicación sin estado: las operaciones de manipulación de recursos a través de peticiones HTTP deben considerarse siempre atómicas. Todas las modificaciones de un recurso deben realizarse dentro de una petición HTTP de forma aislada. Tras la ejecución de la petición, el recurso se deja en un estado final, lo que significa implícitamente que no se admiten actualizaciones parciales del recurso. Siempre se debe enviar el estado completo del recurso [32, pp. 11-12].

En la Figura 14 se ilustra el estilo arquitectónico REST. En ella se puede observar cómo el cliente envía solicitudes HTTP bajo el protocolo de comunicación HTTP 1.1 al servidor. El intercambio de información, generalmente, se realiza en formato JSON.



**Fig. 14.** Estilo arquitectónico REST. Adaptada de [33].

En cada petición, el servidor web devuelve un código de estado HTTP que indica el resultado de la petición. Los códigos de estado HTTP que pueden ser devueltos como respuesta del servidor al llamar a una API RESTful están compuestos por tres dígitos y se dividen en categorías según el primer dígito [34, pp. 60-62], tal y como se describe en la Tabla 1.

**Tabla 1.** Explicación de los códigos HTTP.

| Código de estado | Categoría          | Resultados                                                                   |
|------------------|--------------------|------------------------------------------------------------------------------|
| 1XX              | Informativo        | El servidor ha recibido la petición y procederá con ella.                    |
| 2XX              | Éxito              | El servidor ha recibido, entendido y procesado la petición correctamente.    |
| 3XX              | Redirección        | El servidor ha recibido la solicitud, pero hay una redirección a otra parte. |
| 4XX              | Error del cliente  | La solicitud tiene un error del lado del cliente.                            |
| 5XX              | Error del servidor | El cliente ha realizado una solicitud válida, pero el servidor ha fallado.   |

Los principales objetivos del estilo arquitectónico REST son los siguientes:

- Separación de la representación y el recurso: un recurso no es más que un conjunto de información que puede tener múltiples representaciones; sin embargo, su estado es atómico. Corresponde al emisor de la llamada especificar el tipo de medio deseado con la cabecera *accept* en la petición HTTP, y luego corresponde a la aplicación del servidor gestionar la representación en consecuencia y devolver el tipo de contenido apropiado del recurso y un código de estado HTTP pertinente [32, pp. 13-14].
- Visibilidad: REST está diseñado para ser visible y sencillo. Visibilidad del servicio significa que cada aspecto del mismo debe ser autodescriptivo y seguir el lenguaje HTTP natural [32, p. 14].
- Fiabilidad: por un lado, se considera que un método HTTP es seguro siempre que, cuando se solicite, no modifique ni cause efectos secundarios en el estado del recurso. Por otro lado, se considera que un método HTTP es idempotente si su respuesta es siempre la misma, independientemente del número de veces que se solicite. Por lo tanto, basándose en estas definiciones, el único método seguro es GET; sin embargo, los métodos GET, PUT y DELETE sí que se pueden considerar idempotentes [32, p. 15].
- Escalabilidad: en la WWW (World Wide Web) es esencial que las aplicaciones no tengan estado para ser escalables, por lo que REST es una alternativa que se puede escalar sin problemas [32, p. 15].



- Rendimiento: el rendimiento se mide por el tiempo necesario para procesar una sola solicitud, no por el número total de solicitudes que puede gestionar la aplicación. En general, se considera que REST tiene un buen rendimiento en la mayoría de sistemas [32, p. 16].

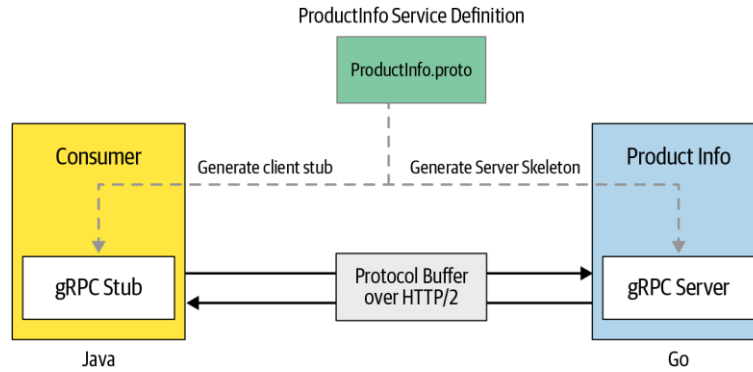
### 2.2.2 El marco de trabajo gRPC

gRPC es un *framework* (marco de trabajo) moderno de código abierto para llamadas a procedimientos remotos que puede ejecutarse en cualquier entorno y permite conectar servicios de forma eficiente dentro de centros de datos y entre ellos, con soporte modular para balanceo de carga, trazabilidad, verificación del estado del servicio y autenticación. También es aplicable en la etapa final de la computación distribuida, para conectar dispositivos, aplicaciones móviles y navegadores con servicios *backend* [35].

Cuando se desarrolla una aplicación gRPC, lo primero que se hace es definir una interfaz de servicio. Esta definición contiene información sobre cómo puede ser consumido el servicio por los clientes: qué métodos pueden llamar de forma remota, qué parámetros deben usarse, qué formatos de mensaje se aplican, etc. El lenguaje utilizado para definir esta interfaz se conoce como lenguaje de definición de interfaces (IDL) [36, p. 3].

A partir de esa definición de servicio, se puede generar el código del lado del servidor, conocido como *server skeleton* o esqueleto del servidor, que simplifica la lógica del lado del servidor al proporcionar abstracciones para la comunicación de bajo nivel. También se puede generar el código del lado del cliente, conocido como *client stub* o cliente auxiliar, que simplifica la comunicación del cliente ocultando los detalles de la comunicación en diferentes lenguajes de programación. Los métodos que se definen en la interfaz de servicio pueden ser invocados remotamente desde el cliente con la misma facilidad que si se tratara de una llamada local a una función. El *framework* subyacente de gRPC se encarga de toda la complejidad normalmente asociada con el cumplimiento estricto de contratos de servicio, la serialización de datos, la comunicación en red, la autenticación, el control de acceso o la observabilidad, entre otros aspectos [36, p. 3].

Para comprender los conceptos fundamentales de gRPC, veamos un caso de uso real de un microservicio implementado con gRPC. Supongamos que estamos construyendo una aplicación de comercio minorista en línea compuesta por múltiples microservicios. Como se ilustra en la Figura 15, imaginemos que queremos crear un microservicio que proporcione los detalles de los productos disponibles en nuestra aplicación de venta en línea. El servicio *ProductInfo* está modelado de tal forma que se expone a través de la red como un servicio gRPC [36, p. 3].



**Fig. 15.** Un microservicio y un consumidor basado en gRPC [36, p. 3].

La definición del servicio se especifica en el archivo *ProductInfo.proto*, el cual es utilizado tanto por el lado del servidor como por el del cliente para generar el código correspondiente. En este ejemplo, se ha asumido que el servicio está implementado en el lenguaje Go y que el consumidor está implementado en Java. La comunicación en red entre el servicio y el consumidor se realiza mediante el protocolo HTTP/2 [36, pp. 3-4].

gRPC utiliza búferes de protocolo (Protocol Buffers) como el lenguaje de definición de interfaces para definir la interfaz del servicio. Protocol Buffers es un mecanismo para serializar datos estructurados que es independiente del lenguaje de programación, neutral respecto a la plataforma y extensible. La definición de la interfaz del servicio se especifica en un archivo proto (archivo de texto con la extensión *.proto*). Los servicios gRPC se definen utilizando el formato común de Protocol Buffers, en el cual los parámetros y tipos de retorno de los métodos RPC se especifican como mensajes de Protocol Buffers. Dado que la definición del servicio es una extensión de la especificación de Protocol Buffers, se utiliza un complemento especial de gRPC para generar código a partir del archivo proto. En nuestro ejemplo, la interfaz del servicio *ProductInfo* puede definirse como se muestra en la Figura 16 [36, p. 4].

```
// ProductInfo.proto
syntax = "proto3"; 1
package ecommerce; 2

service ProductInfo { 3
  rpc addProduct(Product) returns (ProductID); 4
  rpc getProduct(ProductID) returns (Product); 5
}

message Product { 6
  string id = 1; 7
  string name = 2;
  string description = 3;
}

message ProductID { 8
  string value = 1;
}
```

**Fig. 16.** Ejemplo de archivo proto [36, p. 4].

Un servicio es, por tanto, una colección de métodos que pueden ser invocados de forma remota. En la definición de Protocol Buffers, cada método posee parámetros de entrada y tipos de retorno que se establecen como parte del servicio o que pueden importarse. Los parámetros de entrada y retorno pueden ser tipos definidos por el usuario o tipos conocidos de Protocol Buffers definidos en la especificación del servicio. Estos tipos se estructuran como mensajes, donde cada mensaje es un pequeño registro lógico de información que contiene una serie de pares nombre-valor llamados campos. Estos campos tienen números de campo únicos que se usan para identificar los campos en el formato binario del mensaje. Esta definición de servicio se utiliza para construir tanto el lado del servidor como el del cliente de una aplicación gRPC [36, p. 5].

## 2.3 Literatura científica relacionada

### 2.3.1 Revisión de la literatura

Para situar los antecedentes previos al trabajo, se ha realizado una revisión exhaustiva (aunque no sistemática, ya que eso quedaría fuera del ámbito de este TFM) de la literatura científica disponible. Se han encontrado varios estudios que comparan REST con gRPC desde diferentes perspectivas, así como investigaciones que abordan la relación entre el rendimiento y el consumo energético en la comunicación entre microservicios, pero destacaremos [20], [21], [22], [23], [24], [25], [26], [27] y [28] por ser los que mayor relación temática tienen con este estudio y ser todos ellos de reciente actualidad.

Cabe destacar que todos los estudios mencionados se han encontrado en foros académicos y no en literatura gris.

En [20] Zhao et al. analizaron cómo impacta la granularidad de los microservicios en el consumo energético y el rendimiento. Determinaron que la granularidad influye significativamente en el consumo de energía, resultando que un mayor número de servicios suele implicar un mayor uso de energía; en cuanto al tiempo de respuesta, la granularidad tuvo un impacto constante pero reducido, siendo que una granularidad más gruesa generalmente ofrecía un mejor rendimiento.

En [21] se investigó la influencia de los lenguajes de programación Java y Go, así como las arquitecturas de comunicación REST y gRPC, en el rendimiento de las APIs. Soares y Terra definieron el *benchmark* como sigue:

- Parámetros de medida: tiempo medio de respuesta.
- Escenarios: *payload* estándar de 0.763KB y *payload* grande de 781KB.
- Parámetros variables en cada escenario: en cada escenario se hicieron pruebas con los pares Java-REST, Java-gRPC, Go-REST y Go-gRPC.

Los experimentos se llevaron a cabo en el entorno local con el sistema operativo Ubuntu. Por un lado, para un *payload* estándar, los pares Java-gRPC y Go-gRPC fueron los que mejores resultados obtuvieron; por otro lado, para un *payload* grande, los pares Java-REST y Go-REST tuvieron mejor rendimiento. Según los autores, la arquitectura influye un 94% en el rendimiento y el lenguaje solo un 2%.

Niswar et al. [22] evaluaron el rendimiento de la comunicación de microservicios con REST, GraphQL y gRPC. Definieron el siguiente *benchmark*:

- Parámetros de medida: tiempo medio de respuesta y utilización de CPU.
- Escenarios: usuarios concurrentes (no se especifica el número) y usuarios secuenciales en una prueba con una duración determinada de 5 minutos.
- Parámetros variables en cada escenario: número de peticiones (100, 200, 300, 400, 500) y método de recuperación de datos (datos planos y anidados).

Las aplicaciones utilizadas para llevar a cabo los experimentos se ejecutaron en contenedores, pero no se especifica en qué entorno se ejecutaron esos contenedores. Los resultados expusieron que el tiempo de respuesta más rápido lo tuvo gRPC y que REST tuvo menor utilización de CPU.

En [23] los autores investigaron los aspectos prácticos de la evaluación de la eficiencia y el rendimiento de los mecanismos de comunicación REST y gRPC en sistemas basados en microservicios. El *benchmark* se definió tal que así:

- Parámetros de medida: latencia, *throughput* y utilización de CPU.

- Escenarios: petición-respuesta simple, transmisión de datos y manejo de *payloads* grandes.
- Parámetros variables en cada escenario: se varió el número de usuarios concurrentes, pero no se especifica el número exacto.

Las aplicaciones utilizadas para realizar los experimentos se desplegaron en un entorno distribuido en contenedores Docker, pero no se detalla si los contenedores se ejecutaron en la máquina local o en un servidor remoto. Según los resultados, generalmente, gRPC tiene un rendimiento superior, particularmente en escenarios que implican transmisión de datos y complejas interacciones entre microservicios.

En [24] Olivos y Johansson compararon las arquitecturas de comunicación REST y gRPC. Para ello, definieron el siguiente *benchmark*:

- Parámetros de medida: tiempo medio de respuesta, tamaño de la respuesta y *throughput*.
- Escenarios: transferencia de datos pequeña, típica con cálculo, grande con cálculo y grande sin cálculo.
- Parámetros variables en cada escenario: no se varió ningún parámetro.

Los resultados demostraron que solo había ventajas de rendimiento al usar gRPC en comparación con REST.

En [25] se compararon los estilos arquitectónicos REST, WebSocket, gRPC, GraphQL y SOAP. El *benchmark* definido consistió en:

- Parámetros de medida: tiempo medio de respuesta.
- Escenarios: obtener 100 elementos de base de datos, obtener un único elemento e insertar un elemento.
- Parámetros variables en cada escenario: aplicaciones ejecutadas en contenedores con Docker Desktop en la máquina local y aplicaciones ejecutadas directamente en la máquina local.

Los resultados mostraron que el método de transferencia de datos más eficiente y confiable fue gRPC, mientras que GraphQL resultó ser el mecanismo de comunicación más lento de todos.

Bolanowski et al. [26] analizaron los aspectos prácticos del uso de REST y gRPC para realizar tareas de comunicación en aplicaciones dentro de ecosistemas basados en microservicios. Los autores definieron el siguiente *benchmark*:

- Parámetros de medida: tiempo medio de respuesta.
- Escenarios: devolver una cadena de salida similar a la de entrada, intercambiar datos de tamaño pequeño, intercambiar datos de tamaño medio, obtener un fichero de texto de 455KB y descargar un archivo pdf de 3.4MB.

- Parámetros variables en cada escenario: datos encriptados y sin encriptar.

No se especifica si las aplicaciones para poner en práctica los experimentos se ejecutaron en un entorno local o remoto, solamente se menciona que se desplegaron en un entorno distribuido. Atendiendo a los resultados, se observó que REST fue más óptimo para transferir datos simples de tamaño relativamente pequeño, así como para ficheros de texto. Por su lado, gRPC fue más eficiente para transferir archivos grandes y gran volumen de datos.

En [27] Śliwa y Pańczyk compararon el rendimiento de REST, gRPC y GraphQL como mecanismos de comunicación de APIs. La hipótesis de partida fue que gRPC consumía la menor cantidad de datos y era el más eficiente. Para definir el marco experimental de estableció el siguiente *benchmark*:

- Parámetros de medida: tiempo medio de respuesta, *throughput* y tamaño de los datos (de entrada o de salida).
- Escenarios: crear un registro en base de datos, obtener 3112 registros, obtener 111 registros y obtener 11 registros.
- Parámetros variables en cada escenario: un único usuario lanzando peticiones, 10 usuarios simultáneos y 50 usuarios simultáneos.

Los experimentos se llevaron a cabo en el entorno local con el sistema operativo Windows 10. Finalmente, REST fue el que mejores resultados obtuvo en cuanto al tiempo de respuesta y el *throughput*; sin embargo, en cuanto al volumen de datos procesados en las operaciones de obtención y creación, gRPC fue el vencedor.

En [28] se evaluó el consumo energético de REST, SOAP, Socket y gRPC, al ejecutar algoritmos de diferentes complejidades y distintos tamaños y tipos de entrada. El *benchmark* fue definido tal que así:

- Parámetros de medida: consumo de energía.
- Escenarios: algoritmos Bubble Sort, Heap Sort y Selection Sort.
- Parámetros variables en cada escenario: tamaño de datos de entrada y tipo de datos de entrada.

Los experimentos se ejecutaron en un entorno remoto con sistema operativo Windows 7. Los resultados concluyeron que, en el contexto de la descarga de cómputo en móviles, cuando se trata de ejecución remota, REST es la opción más eficiente; sorprendentemente, gRPC nunca fue la configuración más eficiente.

En la Tabla 2 se comparan de forma resumida todos los estudios analizados, ordenados cronológicamente de más reciente a más antiguo.

**Tabla 2.** Comparación de los artículos revisados.

| Artículo | Objetivos                                                                                              | Resultados                                                                                                                         |
|----------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| [20]     | Analizar cómo impacta la granularidad de los microservicios en el consumo energético y el rendimiento. | La granularidad influye principalmente en el consumo energético.                                                                   |
| [21]     | Investigar la influencia de Java, Go, REST y gRPC en el rendimiento de las APIs.                       | REST fue el más eficiente para <i>payloads</i> grandes y gRPC para <i>payloads</i> estándar.                                       |
| [22]     | Evaluar el rendimiento de la comunicación de microservicios con REST, GraphQL y gRPC.                  | El tiempo de respuesta más rápido lo tiene gRPC. REST muestra una menor utilización de CPU.                                        |
| [23]     | Investigar la eficiencia y rendimiento de REST y gRPC en sistemas basados en microservicios.           | Generalmente, gRPC tiene un rendimiento superior.                                                                                  |
| [24]     | Comparar REST y gRPC.                                                                                  | Solo se observan ventajas de rendimiento al usar gRPC en comparación con REST.                                                     |
| [25]     | Comparar los mecanismos de comunicación REST, WebSocket, gRPC, GraphQL y SOAP.                         | El método de transferencia de datos más eficiente y confiable fue gRPC.                                                            |
| [26]     | Analizar REST y gRPC dentro de ecosistemas basados en microservicios.                                  | REST obtuvo mejores resultados para datos de tamaño pequeño y ficheros de texto y gRPC para archivos y volúmenes de datos grandes. |
| [27]     | Comparar el rendimiento de REST, gRPC y GraphQL como mecanismos de comunicación de APIs.               | REST obtuvo mejores resultados para el tiempo de respuesta y el <i>throughput</i> y gRPC para los datos procesados.                |
| [28]     | Evaluar el consumo de energía de REST, SOAP, Socket y gRPC.                                            | REST es la opción más eficiente. Sorprendentemente, gRPC nunca fue la configuración más eficiente.                                 |

Cabe destacar que todos estos trabajos son relativamente recientes, siendo el más antiguo de 2017, uno del 2021, dos de 2022, uno de 2023, tres de 2024 y uno de 2025.

### 2.3.2 Evaluación

En cuanto al rendimiento, los artículos revisados que comparan REST y gRPC son [21], [22], [23], [24], [25], [26] y [27]. En todos ellos se llega a las conclusiones a partir de un análisis de los resultados utilizando la estadística descriptiva, a excepción de [21], en el que además de realizar un análisis basado en la estadística descriptiva, también se hace un análisis estadístico inferencial para sacar las conclusiones. No obstante, las conclusiones de los diferentes estudios no son del todo consistentes si se comparan entre sí. En teoría, gRPC debería tener mayor rendimiento que REST (especialmente en la transferencia de grandes volúmenes de datos) debido, entre otros factores, a que utiliza Protocol Buffers y HTTP/2 en lugar de JSON y HTTP/1.1. Esta superioridad teórica de gRPC es corroborada en [22], [23], [24], [25] y [26]. Sin embargo, en [27] REST mostró un rendimiento superior en todos los casos, aunque la diferencia con gRPC no fue significativa al manejar grandes volúmenes de datos; por su parte, en [21] REST resultó más eficiente con cargas elevadas, mientras que gRPC mostró mejor desempeño con cargas estándar.

En nuestro caso, en lo referente al rendimiento, llevaremos a cabo un riguroso análisis mediante estadística descriptiva e inferencial para poder extraer conclusiones válidas y con soporte estadístico. Asimismo, esperamos que este estudio sirva para aclarar las contradicciones existentes en los artículos revisados.

Respecto al consumo energético, el único estudio que se ha encontrado que compara REST y gRPC ha sido [28]. En este artículo se comparó el consumo de energía de REST y gRPC en la descarga de computación de aplicaciones móviles, extrayendo las conclusiones sin realizar un análisis de estadística inferencial.

En este trabajo, vamos a comparar el consumo energético de REST y gRPC específicamente en el contexto de aplicaciones basadas en microservicios y, además, evaluaremos los resultados obtenidos mediante estadística inferencial.

Por último, no se han encontrado estudios que analicen específicamente si existe relación entre el tiempo de respuesta y el consumo energético en la comunicación entre microservicios, ya que se presupone que a menor tiempo de respuesta mayor será el consumo de energía. Sin embargo, en [20] Zhao et al. demuestran que, de forma general, un mayor rendimiento implica un mayor consumo energético, pero también mencionan que reducir el consumo de energía en la comunicación entre microservicios no implica necesariamente sacrificar los tiempos de respuesta. En este trabajo, mediante pruebas estadísticas de correlación se examinará la relación existente entre ambas variables.



## 2.4 Formulación de hipótesis

Con base en la revisión y evaluación de la literatura científica realizada previamente, estableceremos las siguientes hipótesis para cada una de las preguntas de investigación:

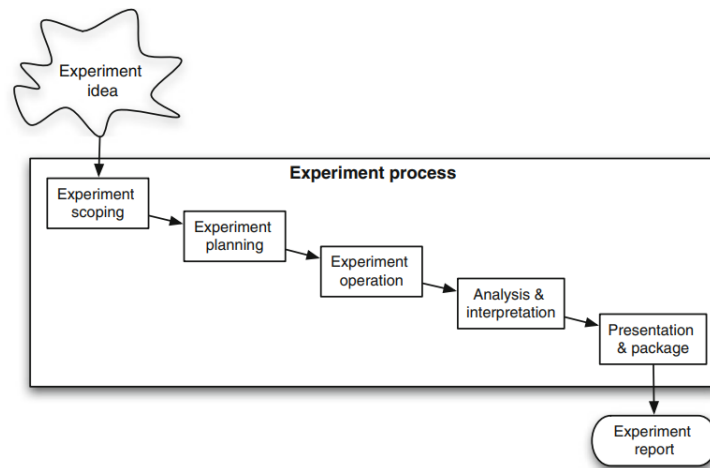
- RQ1: ¿Qué mecanismo de comunicación tiene mejor rendimiento en la obtención de volúmenes de datos de diferente tamaño?
  - Hipótesis: basándonos en la teoría corroborada por [22], [23], [24], [25] y [26] definiremos la hipótesis de que gRPC tiene mejor rendimiento en la obtención de volúmenes de datos de cualquier tamaño, especialmente en grandes volúmenes de datos.
- RQ2: ¿Qué mecanismo de comunicación consume menos energía en la obtención de volúmenes de datos de diferente tamaño?
  - Hipótesis: con base en [28] formularemos la hipótesis de que REST consume menos energía que gRPC en la obtención de volúmenes de datos de cualquier tamaño.
- RQ3: ¿Qué mecanismo de comunicación ofrece mejor rendimiento en la creación de volúmenes de datos de diferente tamaño?
  - Hipótesis: por un lado, en [27], cuando se insertó un elemento en base de datos, el tiempo de respuesta de REST fue menor que el de gRPC. Por otro lado, en [25], cuando el entorno experimental se ejecutó en contenedores Docker, REST tuvo mayor rendimiento, pero cuando se ejecutó en Windows gRPC rindió mejor. Así pues, estableceremos la hipótesis de que REST ofrece mejor rendimiento en la creación de datos, especialmente para volúmenes de datos pequeños y estándar.
- RQ4: ¿Qué mecanismo de comunicación consume menos energía en la creación de volúmenes de datos de diferente tamaño?
  - Hipótesis: de acuerdo con [28] la hipótesis que propondremos es que REST consume menos energía que gRPC en la creación de datos de cualquier tamaño.
- RQ5: ¿Existe relación entre el rendimiento y el consumo de energía en la comunicación entre microservicios?
  - Hipótesis: basándonos en [20] estableceremos la hipótesis de que existe relación entre el rendimiento y el consumo de energía en la comunicación entre microservicios.

### 3 Metodología de desarrollo del entorno experimental

Un experimento en Ingeniería del Software es una investigación empírica que manipula una variable (denominada independiente o factor) del entorno o fenómeno estudiado midiendo el efecto que tiene sobre otra variable denominada variable dependiente. Existen dos tipos de experimentos, los controlados en los que los tratamientos se asignan a los sujetos de manera aleatoria; y los denominados cuasi experimentos, cuando esta aleatorización no es posible. De aquí en adelante hablaremos en términos genéricos de experimentos, refiriéndonos indistintamente a ambos tipos de experimentos (controlados y cuasi). Además, los experimentos pueden ser "orientados a las personas" (*human-oriented experiments*) u "orientados a la tecnología" (*technology-oriented experiments*). En los primeros los sujetos aplican diferentes tratamientos a los objetos; por ejemplo, dos métodos de inspección se aplican sobre dos códigos fuente. En los "orientados a la tecnología", generalmente, se aplican diferentes herramientas a diferentes objetos; por ejemplo, al mismo programa se le aplican dos técnicas de generación de casos de prueba diferentes [37, p. 79].

La principal fortaleza de los experimentos radica en su capacidad para investigar en qué condiciones determinadas afirmaciones se sostienen, y para identificar en qué contextos resultan útiles ciertos estándares, métodos o herramientas. En todos los experimentos se persigue la comprobación de hipótesis: si los resultados obtenidos contradicen una hipótesis formulada, esta puede ser rechazada. Para llevar a cabo un experimento, es necesario seguir un proceso experimental que describa las actividades a realizar, junto con sus respectivas entradas y salidas. Este proceso no es secuencial en cascada, ya que no requiere completar una actividad para iniciar la siguiente. De hecho, el orden indicado refleja únicamente el punto de inicio de cada actividad, sin implicar una ejecución estrictamente lineal. Se trata de un proceso iterativo, en el que puede ser necesario retroceder para refinar actividades previas antes de continuar. No obstante, una vez iniciada la ejecución del experimento, ya no es posible modificar la definición de objetivos ni la planificación [37, pp. 79-80].

El proceso experimental propuesto por Wohlin et al. [38], representado en la Figura 17, consta de cinco actividades: definición del alcance, planificación, operación, análisis e interpretación y presentación y difusión. A su vez, cada actividad tiene sus correspondientes tareas. Este trabajo adopta dicho proceso como base metodológica, aunque no se sigue de forma completamente rigurosa, sino adaptado a las necesidades del estudio.



**Fig. 17.** Resumen del proceso experimental [38, p. 77].

La definición del alcance es el primer paso, donde se delimita el experimento en términos del problema, el objetivo y las metas. A continuación, viene la planificación, donde se determina el diseño del experimento, se consideran los instrumentos necesarios y se evalúan las amenazas que podrían afectarlo. La operación del experimento se deriva del diseño. En esta fase operativa, se recolectan las mediciones, las cuales son posteriormente analizadas y evaluadas durante la fase de análisis e interpretación. Finalmente, los resultados se presentan y empaquetan en la fase de presentación y difusión [38, p. 77].

En este trabajo, con el fin de realizar los experimentos, se desarrollarán dos entornos basados en arquitectura de microservicios: uno con mecanismo de comunicación REST y otro con gRPC. Se tomará como punto de partida el entorno empleado en [22], adaptándolo a nuestras necesidades e incluyendo algunas mejoras, como la incorporación de Docker, herramienta más usada en la actualidad para la contenedorización de aplicaciones. Cada entorno experimental implementará las mismas aplicaciones, que se ejecutarán en contenedores. Concretamente, cada entorno tendrá un microservicio de autenticación y un microservicio para gestionar usuarios. Este último estará conectado a una base de datos relacional.

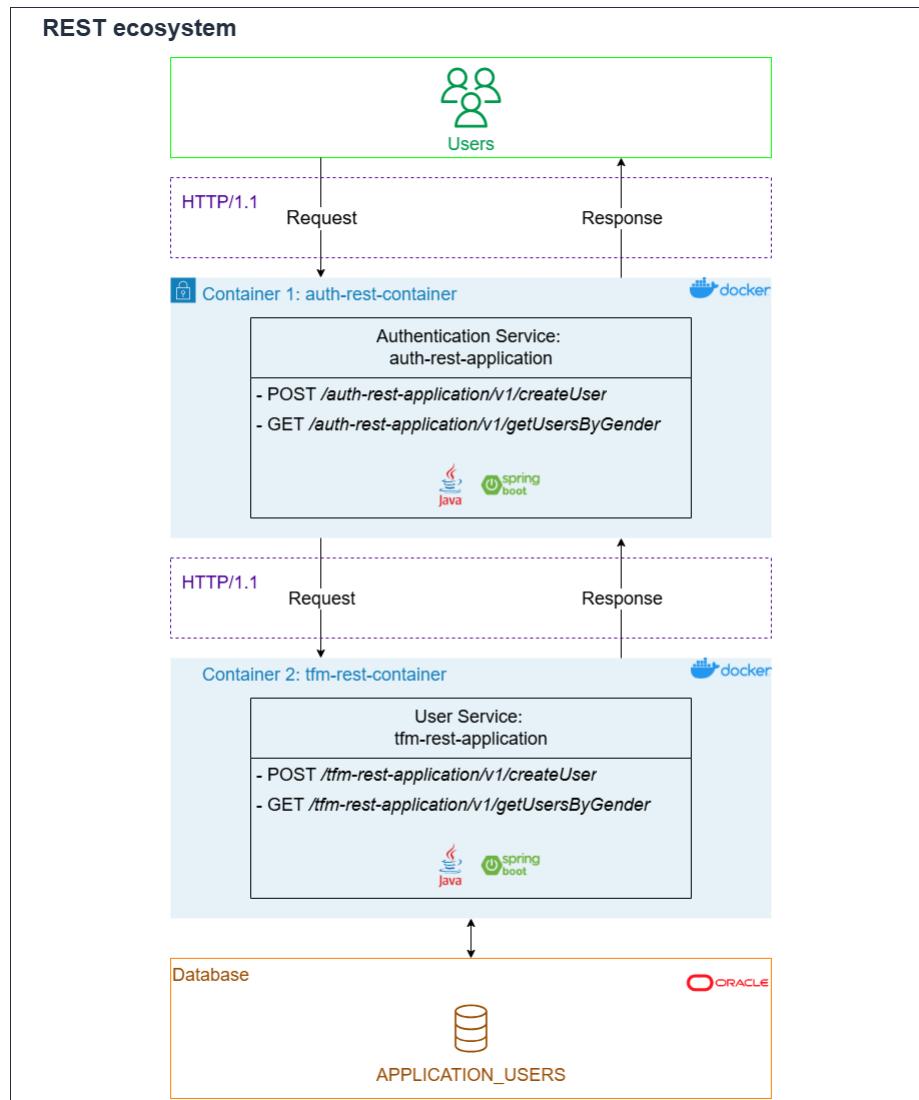
### 3.1 Diseño del ecosistema basado en microservicios

La base de datos, común a ambos ecosistemas, contendrá únicamente una tabla denominada APPLICATION\_USERS. Para implementar la base de datos se usará Oracle Database 21c Express Edition (XE) [39], ya que es ideal para aplicaciones que se ejecutan en local y manejan un volumen de datos pequeño. Además, Oracle XE es gratuita.

Grandes y conocidas empresas han informado que, al contenedorizar aplicaciones heredadas existentes (muchas veces llamadas aplicaciones tradicionales) y establecer una cadena de suministro de software totalmente automatizada basada en contenedores, pueden reducir el coste de mantenimiento de esas aplicaciones críticas para el negocio en un 50% a 60%, y pueden reducir el tiempo entre nuevos lanzamientos de estas aplicaciones tradicionales hasta en un 90%. En este sentido, Docker se ha convertido en un referente, siendo la tecnología más utilizada [40, p.16]. En 2016 Microsoft creó una implementación nativa de Docker, siguiendo las recomendaciones de Open Container Initiative. De esta forma, se consiguió la compatibilidad de Docker con Windows [41, p. 457]. Así pues, con el objetivo de crear entornos experimentales lo más realistas posible y replicables, tanto en el ecosistema REST como en el basado en gRPC, los microservicios se ejecutarán en contenedores Docker, concretamente usando la versión 28.0.4 [42].

### **3.1.1 Ecosistema con mecanismo de comunicación REST**

En la Figura 18 se muestra la arquitectura del ecosistema con mecanismo de comunicación REST.

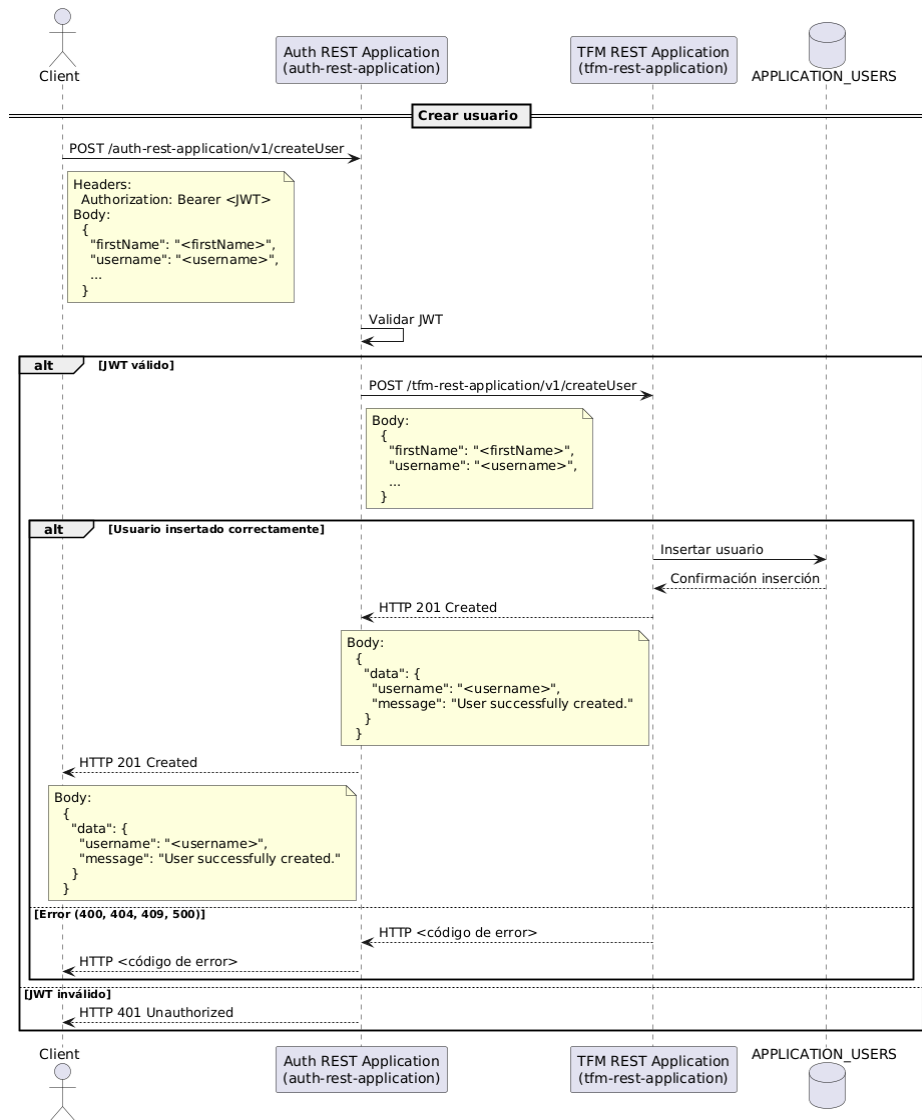


**Fig. 18.** Arquitectura del ecosistema con mecanismo de comunicación REST.

Para definir las interfaces REST de los microservicios, usaremos la versión 3.0.0 de la especificación OpenAPI. La especificación OpenAPI (OAS) define una interfaz estándar e independiente del lenguaje para APIs RESTful, que permite tanto a humanos como a sistemas informáticos descubrir y comprender las capacidades del servicio sin necesidad de acceder al código fuente, a la documentación o tener que inspeccionar el tráfico de red. Cuando está bien definida, un consumidor puede entender e interactuar con el servicio remoto con una cantidad mínima de lógica de implementación [43].

Por un lado, tenemos el microservicio *auth-rest-application* (ejecutado en el contenedor *auth-rest-container*), que implementará los *endpoints* “POST /*auth-rest-application/v1/createUser*” y “GET /*auth-rest-application/v1/getUsersByGender*”. Por otro lado, está el microservicio *tfm-rest-application* (ejecutado en el contenedor *tfm-rest-container*), que implementará los *endpoints* “POST /*tfm-rest-application/v1/createUser*” y “GET /*tfm-rest-application/v1/getUsersByGender*”.

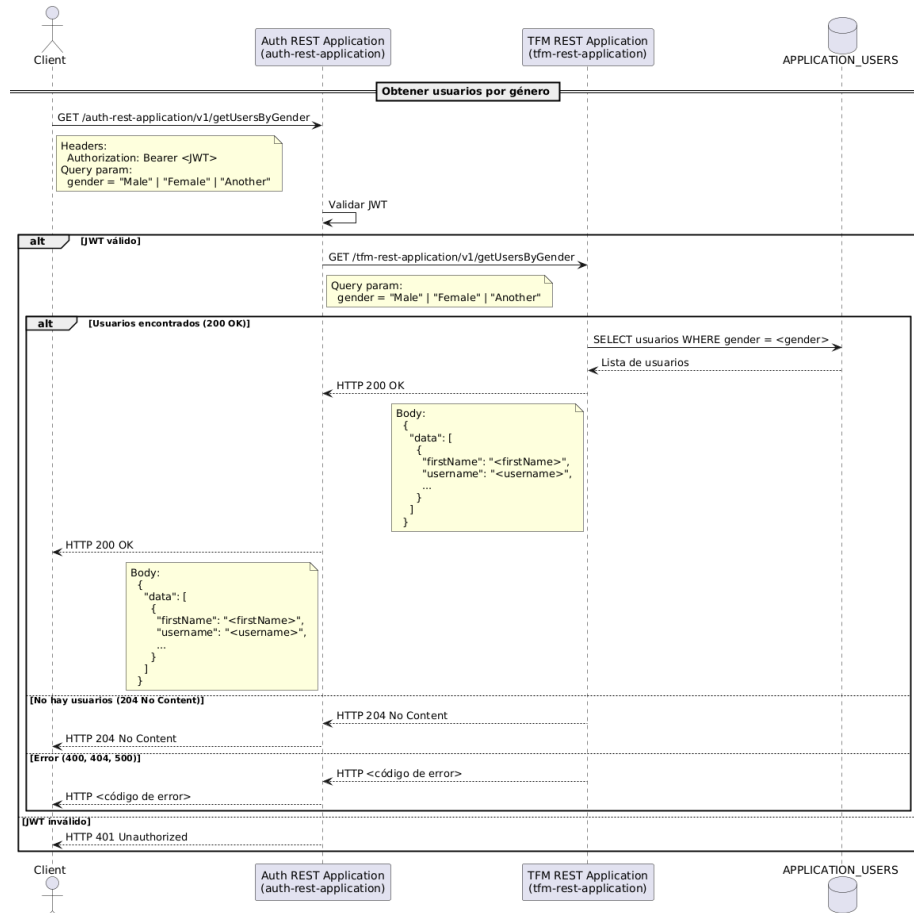
Para crear un nuevo usuario en base de datos, el cliente enviará una petición, usando el protocolo HTTP/1.1, a *auth-rest-application*, llamando al *endpoint* “POST /*auth-rest-application/v1/createUser*”. En la llamada, el cliente mandará en el cuerpo de la petición el JSON para crear el usuario y en las cabeceras incluirá el *header* “Authorization” con el Bearer Token en formato JWT (JSON Web Token). La aplicación *auth-rest-application* validará el token y redirigirá la petición, vía HTTP/1.1, al *endpoint* “POST /*tfm-rest-application/v1/createUser*” de *tfm-rest-application*, que se encargará de guardar en base de datos el usuario. Si el usuario se ha guardado satisfactoriamente, *tfm-rest-application* responderá con un código el estado HTTP “201 Created” y un JSON en el cuerpo de la respuesta con datos detallados. En la Figura 19 se puede ver un diagrama de secuencia UML detallado.



**Fig. 19.** Diagrama de secuencia UML para crear un usuario en el ecosistema REST.

Para obtener el listado de usuarios filtrando por género de base de datos, el cliente enviará una petición, usando el protocolo HTTP/1.1, a *auth-rest-application*, llamando al endpoint “GET /auth-rest-application/v1/getUsersByGender”. En la llamada, el cliente mandará el *query param* “gender” y en las cabeceras incluirá el *header* “Authorization” con el Bearer Token en formato JWT (JSON Web Token). La aplicación *auth-rest-application* validará el token y redirigirá la petición, vía HTTP/1.1, al endpoint “GET /tfm-rest-application/v1/getUsersByGender” de *tfm-rest-application*, que se encargará de obtener de base de datos la lista de usuarios con el género indicado en el *query param*

“gender”. Si todo funciona correctamente, *tfm-rest-application* responderá con el código de estado HTTP “200 OK” y un JSON en el cuerpo de la respuesta con el listado de usuarios y la información detallada de cada usuario. En la Figura 20 se muestra un diagrama de secuencia UML que representa esto.

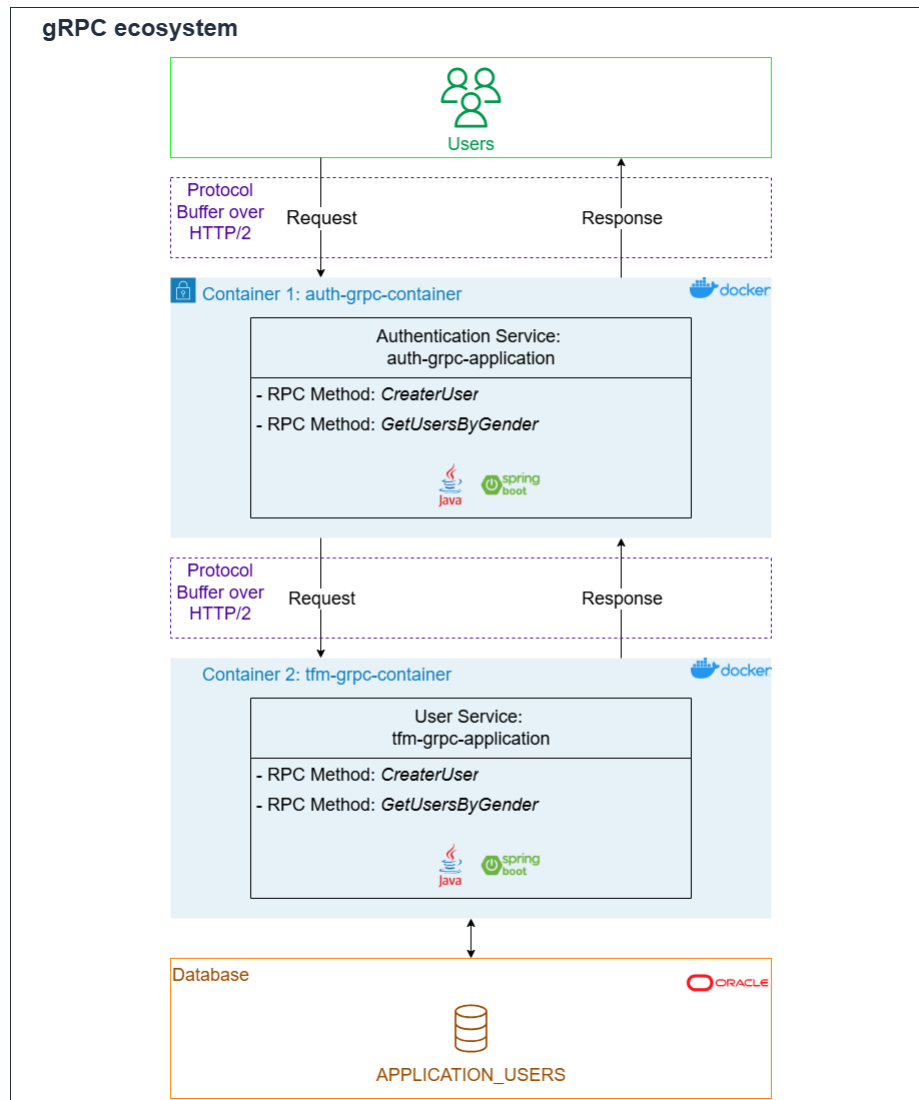


**Fig. 20.** Diagrama de secuencia UML para obtener usuarios por género en el ecosistema REST.

### 3.1.2 Ecosistema con mecanismo de comunicación gRPC

La Figura 21 representa la arquitectura del ecosistema con mecanismo de comunicación gRPC.





**Fig. 21.** Arquitectura del ecosistema con mecanismo de comunicación gRPC.

Para definir las interfaces de los microservicios en gRPC, se utilizará la sintaxis *proto3* de Protocol Buffers [44].

Por un lado, tenemos el microservicio *auth-grpc-application* (ejecutado en el contenedor *auth-grpc-container*), que implementará los métodos RPC *CreateUser* y *GetUsersByGender*. Por otro lado, está el microservicio *tfm-grpc-application* (ejecutado en el contenedor *tfm-grpc-container*), que también implementará los métodos RPC *CreateUser* y *GetUsersByGender*.

Para crear un nuevo usuario en base de datos, el cliente enviará una petición, usando el protocolo HTTP/2, a *auth-grpc-application*, llamando al método RPC *CreateUser*. En el mensaje de la solicitud el cliente mandará la información del usuario y en los metadatos incluirá el campo “Authorization” con el Bearer Token en formato JWT (JSON Web Token). La aplicación *auth-grpc-application* validará el token y redirigirá la petición, vía HTTP/2, al método RPC *CreateUser* de *tfm-grpc-application*, que se encargará de guardar en base de datos el usuario. Si el usuario se ha guardado satisfactoriamente, *tfm-grpc-application* responderá con el código de estado gRPC “0 OK” y con información detallada en formato binario *protobuf*. En la Figura 22 se muestra un diagrama de secuencia UML que representa lo anterior.

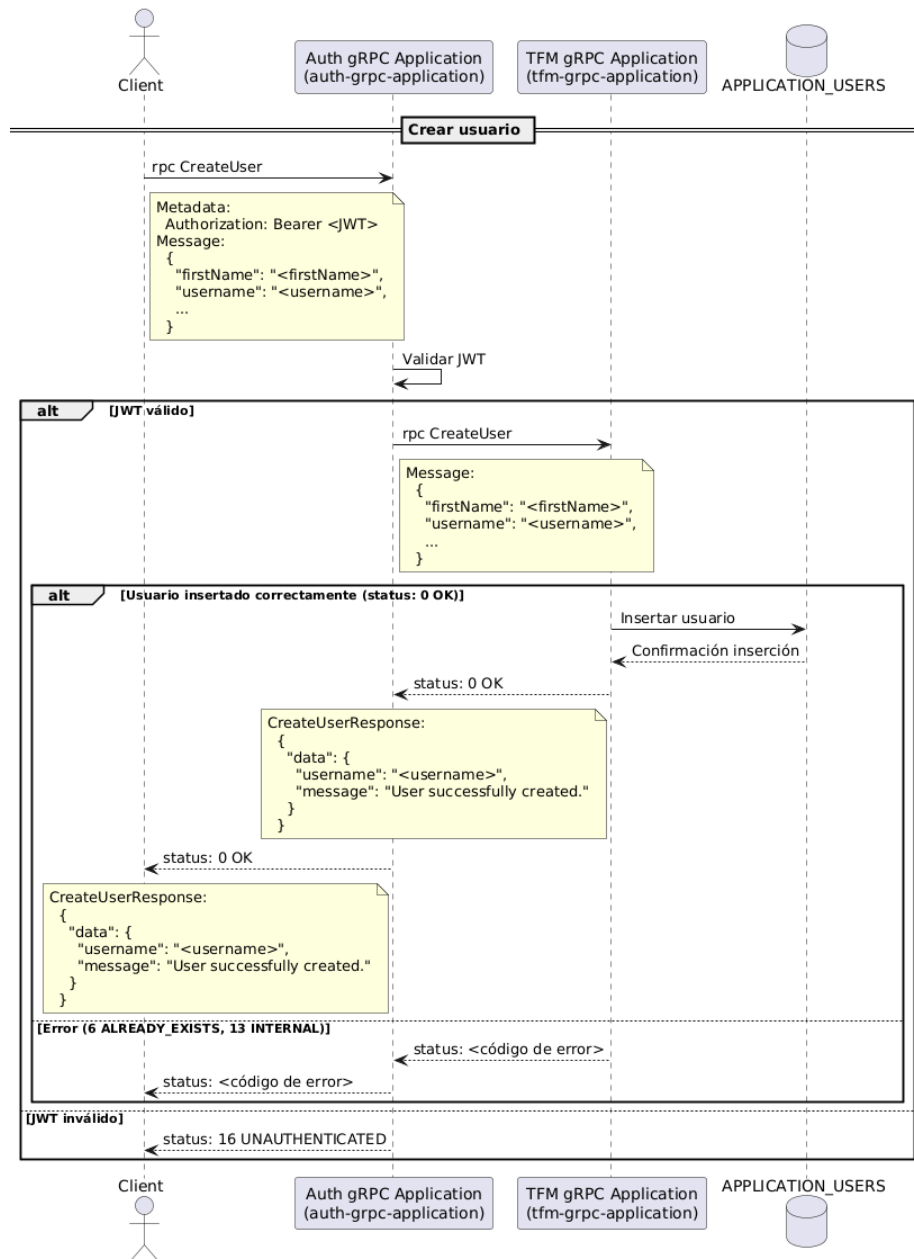


Fig. 22. Diagrama de secuencia UML para crear un usuario en el ecosistema gRPC.

Para obtener el listado de usuarios filtrando por género de base de datos, el cliente enviará una petición, usando el protocolo HTTP/2, a *auth-grpc-application*, llamando al método RPC *GetUsersByGender*. En el mensaje de la solicitud el cliente mandará el campo “gender” y en los metadatos incluirá el campo “Authorization” con el Bearer

Token en formato JWT (JSON Web Token). La aplicación *auth-grpc-application* validará el token y redirigirá la petición, vía HTTP/2, al método RPC *GetUsersByGender* de *tfm-grpc-application*, que se encargará de obtener de base de datos la lista de usuarios con el género indicado en el campo “gender”. Si todo funciona correctamente, *tfm-grpc-application* responderá con el código de estado gRPC “0 OK” y con el listado de usuarios, incluyendo la información detallada de cada usuario, en formato binario *protobuf*. En la Figura 23 se puede ver un diagrama de secuencia UML detallado.

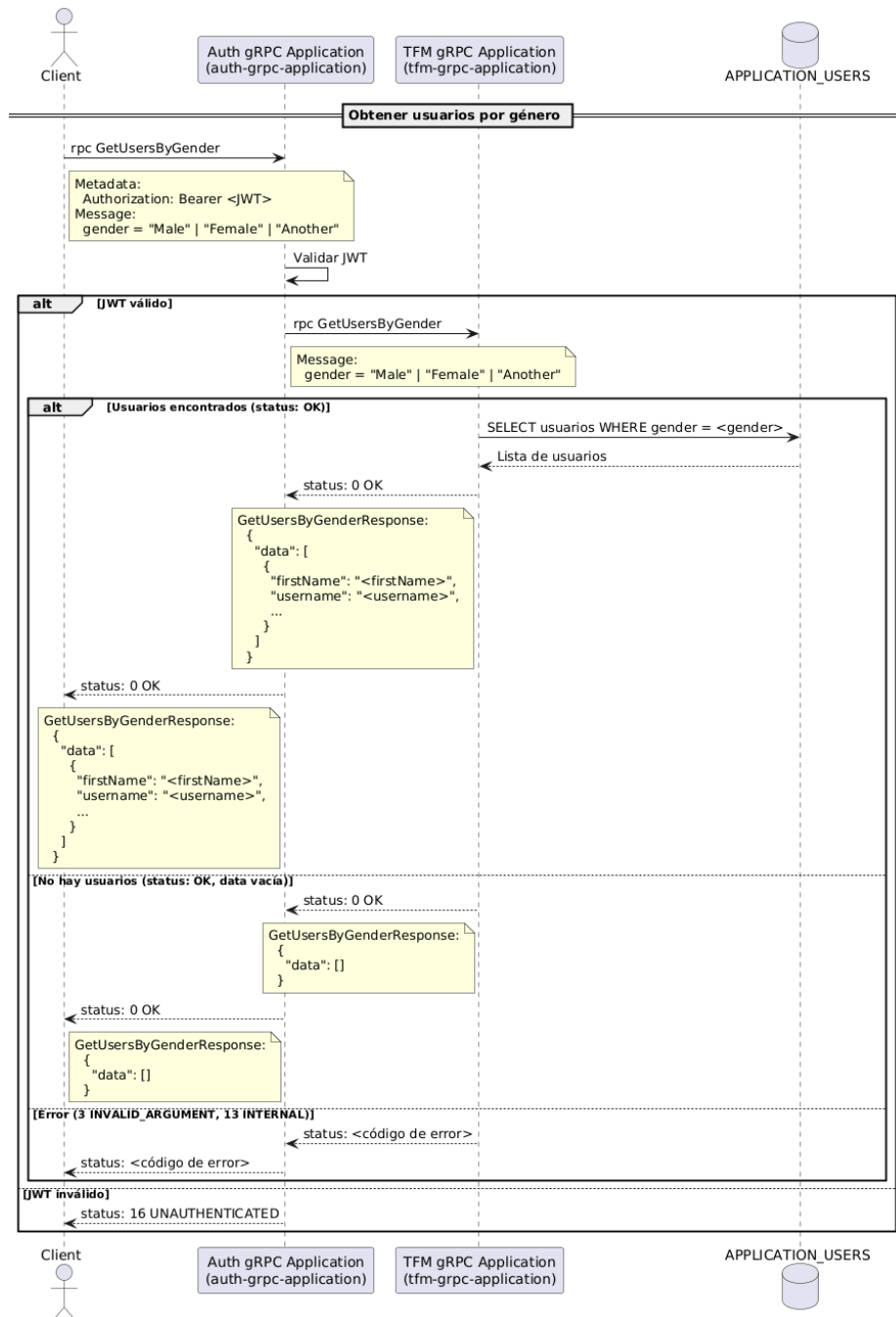


Fig. 23. Diagrama de secuencia UML para obtener usuarios por género en el ecosistema gRPC.

### 3.2 Desarrollo y ejecución del ecosistema basado en microservicios

Para el desarrollo de los microservicios se empleará el entorno de desarrollo integrado (IDE) IntelliJ IDEA Community Edition (versión 2025.1), de uso gratuito [45]. El lenguaje de programación seleccionado será Java 17 [46], mientras que para la construcción de la lógica de negocio y la configuración de los servicios se utilizará el *framework* Spring Boot 3.4.2 [47]. La gestión y automatización del ciclo de vida de los proyectos se llevará a cabo con Apache Maven 3.8.7 [48]. Finalmente, el código fuente se alojará en GitHub, que servirá como plataforma de control de versiones y colaboración [49].

#### 3.2.1 Desarrollo del ecosistema con mecanismo de comunicación REST

A continuación, se presentan los enlaces de GitHub para acceder al código fuente de los repositorios públicos desarrollados en el entorno experimental basado en REST:

- auth-rest-application: <https://goo.su/ZfTpHfr>.
- tfm-rest-application: <https://goo.su/gGHjpdX>.

#### 3.2.2 Desarrollo del ecosistema con mecanismo de comunicación gRPC

Los enlaces de los repositorios públicos de GitHub que contienen el código fuente desarrollado en el entorno experimental basado en gRPC son los siguientes:

- auth-grpc-application: <https://goo.su/EuHSAY>.
- tfm-grpc-application: <https://goo.su/gWLnrv>.

#### 3.2.3 Entorno de ejecución

Todos los contenedores Docker y la base de datos se ejecutarán en una máquina local, ya que así conseguimos que los resultados sean más estables al no incluir los retrasos y latencia aleatorios que pueden introducir las conexiones a máquinas externas a través de Internet. Las características de la máquina local se muestran en la Tabla 3.

**Tabla 3.** Características del entorno de ejecución.

| Categoría            | Especificación                                   |
|----------------------|--------------------------------------------------|
| Marca                | Lenovo                                           |
| Modelo               | ThinkPad E560                                    |
| Procesador           | Intel ® Core ™ i5-6200U<br>CPU @ 2.30Ghz 2.40GHz |
| Núcleos              | 2 ( <i>dual-core con hyper-threading</i> )       |
| Procesadores lógicos | 4                                                |
| Memoria RAM          | 8GB                                              |
| Memoria disco duro   | 450GB                                            |
| Sistema operativo    | Windows 10 Pro (64 bits)                         |

## 4 Metodología experimental

### 4.1 Elección de herramientas de medición

Por un lado, para llevar a cabo los experimentos y medir el rendimiento se utilizará Apache JMeter [50], ya que es una herramienta ampliamente aceptada en la industria y de uso gratuito.

Por otro lado, para medir el consumo de energía se utilizará Intel Power Gadget [51], ya que es la única herramienta que se ha encontrado que puede medir la energía en el sistema operativo Windows. En nuestra máquina, al tener un procesador Intel, la podemos usar sin problema. Intel Power Gadget mide la energía total consumida por los núcleos de la CPU durante un periodo de tiempo.

### 4.2 Definición del marco experimental

Las variables categóricas e independientes de los experimentos serán los ecosistemas basados en REST y gRPC, es decir, todos los experimentos se ejecutarán en ambos entornos con el objetivo final de comparar los resultados. Por su parte, las variables numéricas y dependientes serán los parámetros de medida descritos en el apartado 4.2.1.

En cada uno de los escenarios (definidos en el apartado 4.2.2) se realizarán experimentos con 100, 300 y 500 muestras. Cada muestra representa una petición del cliente al servidor.

#### 4.2.1 Parámetros de medida

Los parámetros de medida seleccionados serán los siguientes:

- Tiempo de respuesta: mide cuánto tarda una petición desde que se envía hasta que se recibe completamente la respuesta.
- *Throughput*: indica cuántas solicitudes se han procesado por unidad de tiempo.
- Consumo de energía: mide la energía consumida por los núcleos de la CPU.

Para evaluar el rendimiento, el principal parámetro de medida que utilizaremos será el tiempo de respuesta y, adicionalmente, el *throughput*. Por su parte, para evaluar el consumo energético usaremos el consumo de energía de los núcleos de la CPU.

#### 4.2.2 Escenarios

Para definir los escenarios, estableceremos tres tamaños diferentes de datos, a los cuales, en adelante, denominaremos “pequeño”, “estándar” y “grande” para simplificar. Según [52], la mediana de los datos transmitidos en solicitudes HTTP comúnmente observadas



en aplicaciones web es de 1463 *bytes*, mientras que la mediana de los datos recibidos es de 1643 *bytes*. Con base en esto, definiremos los siguientes escenarios, tomando como referencia el tamaño del *payload* en formato JSON:

#### 4.2.2.1 Obtención de datos

- Volúmenes de datos pequeños: *payload* de respuesta de 0,18KB.
- Volúmenes de datos estándar: *payload* de respuesta de 1,59KB.
- Volúmenes de datos grandes: *payload* de respuesta de 814,84KB.

#### 4.2.2.2 Creación de datos

- Volúmenes de datos pequeños: *payload* de entrada de 0,25KB
- Volúmenes de datos estándar: *payload* de entrada de 1,54KB.
- Volúmenes de datos grandes: *payload* de entrada de 801,95KB.

En investigación, siempre que se puede, se utilizan datos generados aleatoriamente dentro de un rango para que los resultados sean más generalizables. No obstante, en este trabajo hemos decidido que el tamaño de los *payloads* sea fijo, siguiendo una metodología similar a la de los estudios revisados. De este modo, se facilita la comparación entre nuestros resultados y los de las investigaciones revisadas previamente, siendo así más directa y precisa. Asimismo, generar datos con tamaños completamente aleatorios no aportaría un valor significativo al estudio, ya que en la práctica las aplicaciones no producen cargas de manera totalmente aleatoria, sino que los datos suelen concentrarse en intervalos de tamaño relativamente acotados.

### 4.3 Ejecución de los experimentos

A continuación, para cada uno de los experimentos ejecutados se muestra un resumen de los resultados obtenidos. El tiempo medio de respuesta es el promedio del tiempo que tarda el servidor en responder a una solicitud, calculado a partir del total de muestras enviadas durante la prueba, medido en milisegundos (ms). Por otro lado, el *throughput* representa el número de solicitudes que se han procesado por unidad de tiempo durante el experimento, medido en peticiones/s. Finalmente, el consumo energético medio se ha calculado dividiendo el consumo de energía total de la prueba entre el número de muestras, medido en julios (J).

En el siguiente enlace se pueden ver los informes de rendimiento de JMeter y de consumo de energía de Intel Power Gadget obtenidos en cada uno de los escenarios: <https://goo.su/Bi7X8J>.

#### 4.3.1 Experimentos para peticiones de obtención de datos

En la Tabla 4 se puede ver un resumen de los resultados (tiempo medio de respuesta, *throughput* y consumo medio de energía) conseguidos en los experimentos de obtención de datos de volúmenes pequeño, estándar y grande. Para cada tamaño de datos se han realizado seis experimentos: REST con 100 muestras, gRPC con 100 muestras, REST con 300 muestras, gRPC con 300 muestras, REST con 500 muestras y gRPC con 500 muestras,

**Tabla 4.** Resultados de los experimentos de obtención de datos.

|                             |              |      | Tiempo medio de respuesta | Throughput | Consumo medio de energía |
|-----------------------------|--------------|------|---------------------------|------------|--------------------------|
| Volúmenes de datos pequeños | 100 muestras | REST | 2397ms                    | 25/min     | 6,273J                   |
|                             |              | gRPC | 2409ms                    | 24,9/min   | 6,037J                   |
|                             | 300 muestras | REST | 2388ms                    | 25,1/min   | 5,801J                   |
|                             |              | gRPC | 2431ms                    | 24,7/min   | 5,927J                   |
|                             | 500 muestras | REST | 2439ms                    | 24,6/min   | 6,192J                   |
|                             |              | gRPC | 2457ms                    | 24,4/min   | 5,986J                   |
| Volúmenes de datos estándar | 100 muestras | REST | 2534ms                    | 23,7/min   | 6,503J                   |
|                             |              | gRPC | 2756ms                    | 21,7/min   | 6,945J                   |
|                             | 300 muestras | REST | 2524ms                    | 23,8/min   | 7,059J                   |
|                             |              | gRPC | 2667ms                    | 22,5/min   | 6,548J                   |
|                             | 500 muestras | REST | 2511ms                    | 23,9/min   | 7,935J                   |
|                             |              | gRPC | 2708ms                    | 22,1/min   | 6,619J                   |
| Volúmenes de datos grandes  | 100 muestras | REST | 3146ms                    | 19,1/min   | 10,793J                  |
|                             |              | gRPC | 3130ms                    | 19,1/min   | 10,186J                  |
|                             | 300 muestras | REST | 3057ms                    | 19,6/min   | 10,451J                  |
|                             |              | gRPC | 2766ms                    | 21,7/min   | 9,313J                   |
|                             | 500 muestras | REST | 2867ms                    | 20,9/min   | 10,101J                  |
|                             |              | gRPC | 2844ms                    | 21,1/min   | 9,675J                   |

#### 4.3.2 Experimentos para peticiones de creación de datos

En la Tabla 5 se pueden observar los resultados obtenidos en los experimentos de creación de datos de volúmenes pequeño, estándar y grande.

**Tabla 5.** Resultados de los experimentos de creación de datos.

|                             |              |      | Tiempo medio de respuesta | Throughput | Consumo medio de energía |
|-----------------------------|--------------|------|---------------------------|------------|--------------------------|
| Volúmenes de datos pequeños | 100 muestras | REST | 2975ms                    | 20,2/min   | 10,676J                  |
|                             |              | gRPC | 4028ms                    | 14,9/min   | 10,53J                   |
|                             | 300 muestras | REST | 3119ms                    | 19,2/min   | 10,439J                  |
|                             |              | gRPC | 3992ms                    | 15/min     | 10,568J                  |
|                             | 500 muestras | REST | 2877ms                    | 20,8/min   | 9,83J                    |
|                             |              | gRPC | 4171ms                    | 14,4/min   | 10,868J                  |
| Volúmenes de datos estándar | 100 muestras | REST | 3024ms                    | 19,8/min   | 12,241J                  |
|                             |              | gRPC | 4302ms                    | 13,9/min   | 11,988J                  |
|                             | 300 muestras | REST | 3138ms                    | 19,1/min   | 13,691J                  |
|                             |              | gRPC | 4342ms                    | 13,8/min   | 11,984J                  |
|                             | 500 muestras | REST | 2888ms                    | 20,8/min   | 10,397J                  |
|                             |              | gRPC | 4052ms                    | 14,8/min   | 10,969J                  |
| Volúmenes de datos grandes  | 100 muestras | REST | 5421ms                    | 11,1/min   | 26,044J                  |
|                             |              | gRPC | 4564ms                    | 13,1/min   | 15,4J                    |
|                             | 300 muestras | REST | 5593ms                    | 10,8/min   | 20,783J                  |
|                             |              | gRPC | 4501ms                    | 13,3/min   | 15,223J                  |
|                             | 500 muestras | REST | 5020ms                    | 11,9/min   | 17,424J                  |
|                             |              | gRPC | 4515ms                    | 13,3/min   | 13,54J                   |

## 5 Análisis estadístico

Para evaluar los resultados, se compararán gráficamente los resultados obtenidos en los ecosistemas REST y gRPC en cuanto al tiempo de respuesta, el *throughput* y el consumo energético en cada uno de los escenarios. El tiempo de respuesta se visualizará con un diagrama de cajas y el *throughput* y el consumo energético con un gráfico de barras. Para realizar el análisis estadístico inferencial de forma precisa y realizar las gráficas de forma rápida y sencilla, se ha desarrollado un *notebook* de Jupyter con Python, al cual se puede acceder desde el siguiente enlace: <https://goo.su/Bz8d>.

Con el fin de determinar si existen diferencias significativas entre los entornos REST y gRPC en cuanto al tiempo de respuesta en cada uno de los escenarios, así como analizar si existe una correlación entre el consumo energético y el tiempo de respuesta en la comunicación entre microservicios, se llevará a cabo un análisis estadístico inferencial. Se emplearán diversas pruebas estadísticas y en todas ellas se seguirá el mismo proceso. En primer lugar, se formulará una hipótesis nula ( $H_0$ ) y una hipótesis alternativa ( $H_1$ ) y se fijará un nivel de significancia del 5% ( $\alpha$ ). Después, se calculará el estadístico de prueba o coeficiente de correlación que corresponda, así como su valor de probabilidad ( $p$ ) asociado. Finalmente, si  $p$  es mayor que  $\alpha$  no se rechaza  $H_0$ ; en caso contrario se rechaza  $H_0$  en favor de  $H_1$ .

Por un lado, para verificar si existen diferencias significativas entre los entornos REST y gRPC en el tiempo de respuesta en cada uno de los escenarios, se llevarán a cabo dos pruebas estadísticas. Primeramente, para determinar si ambos conjuntos de datos siguen una distribución normal, se utilizará el test de normalidad de Shapiro-Wilk (1), calculando el estadístico de prueba  $W$ . Después, para concluir si existen diferencias significativas entre REST y gRPC, en el caso de que los dos conjuntos de datos sigan una distribución normal, se aplicará la prueba paramétrica  $t$  de Student (2) y se hallará el estadístico de prueba  $t$ ; por el contrario, si al menos uno de los dos conjuntos de datos no sigue una distribución normal, se aplicará la prueba no paramétrica de la  $U$  de Mann-Whitney (3) y se calculará el estadístico de prueba  $U$ .

$$W = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (1)$$

$$t = \frac{\bar{x} - \mu_0}{s / \sqrt{n}} \quad (2)$$

$$U = R_1 - \frac{n_1(n_1+1)}{2} \quad (3)$$

Cabe reseñar que para el *throughput* no se puede realizar un análisis de inferencia estadística porque no hay un dato para cada muestra, sino que el dato obtenido es del total de cada experimento. En el caso del consumo energético, tampoco tenemos un valor para cada muestra, por lo que hemos hecho una aproximación, calculando el consumo energético medio por petición y asignando este valor a cada una de las muestras; en este caso, al no haber varianza en el conjunto de datos no se puede aplicar ningún test de normalidad ni ningún test de hipótesis.

Por otro lado, con el propósito de evaluar si existe correlación entre el consumo energético y el tiempo de respuesta en la comunicación entre microservicios, se conformarán dos conjuntos de datos: uno con los tiempos de respuesta promedio de todos los escenarios en ambos entornos (REST y gRPC) y otro con los consumos energéticos promedio correspondiente a esos mismos escenarios y entornos. En este caso también se ejecutarán dos pruebas estadísticas. Inicialmente, se comprobará mediante Shapiro-Wilk (1) si ambos conjuntos siguen una distribución normal. Luego, para determinar si existe correlación entre el tiempo de respuesta y el consumo de energía, si ambos conjuntos de datos siguen una distribución normal, se aplicará el coeficiente de correlación de Pearson (4), calculando el coeficiente  $r$ ; en caso contrario, se hallará el coeficiente de correlación de Spearman (5)  $\rho$ .

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (4)$$

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)} \quad (5)$$

## 5.1 Experimentos de obtención de datos

### 5.1.1 Experimentos de obtención de volúmenes de datos pequeños

#### 5.1.1.1 Tiempo de respuesta

- 100 muestras

En primer lugar, se ejecutará la prueba de Shapiro-Wilk para determinar si los conjuntos de datos REST y gRPC siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,886$

- $p_{REST} \approx 0$
- $W_{gRPC} = 0,889$
- $p_{gRPC} \approx 0$

Como  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , rechazamos  $H_0$  y aceptamos  $H_1$ . Así pues, se aplicará la prueba de la U de Mann-Whitney para resolver si existen diferencias significativas entre REST y gRPC en cuanto al tiempo de respuesta.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 4906,5$
  - $p = 0,82$

No se rechaza  $H_0$  porque  $p > \alpha$ . De esta forma, llegamos a la conclusión de que no hay diferencia significativa entre el tiempo de respuesta de REST y gRPC en la obtención de volúmenes de datos pequeños al ejecutar el experimento con 100 muestras.

▪ 300 muestras

Para empezar, se llevará a cabo la prueba de Shapiro-Wilk para determinar si los conjuntos de datos REST y gRPC siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,87$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,723$
  - $p_{gRPC} \approx 0$

En este caso,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  y aceptamos  $H_1$ . Así pues, se aplicará la prueba de la U de Mann-Whitney para resolver si existen diferencias significativas entre REST y gRPC en cuanto al tiempo de respuesta.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$



- $U = 39788$
- $p = 0,014$

Rechazamos  $H_0$  en favor de  $H_1$  porque  $p < \alpha$ . De esta manera, concluimos que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la obtención de volúmenes de datos pequeños cuando el experimento se ejecuta con 300 muestras.

▪ 500 muestras

En primera instancia, se ejecutará la prueba de Shapiro-Wilk para ver si los conjuntos de datos REST y gRPC siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,858$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,792$
  - $p_{gRPC} \approx 0$

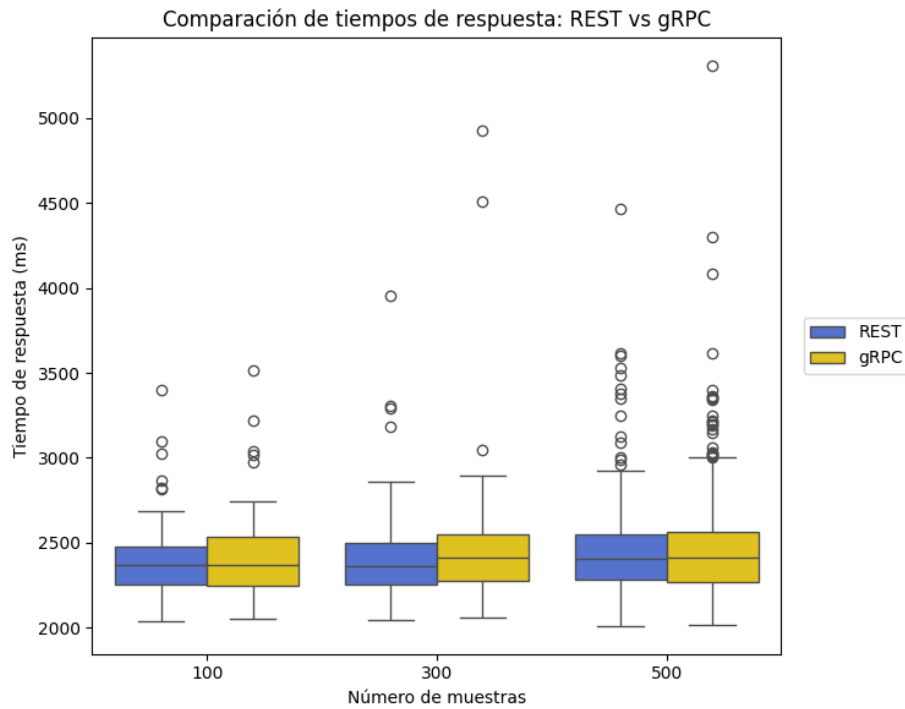
Al realizar el experimento con 500 muestras, apreciamos que  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , así que rechazamos  $H_0$  y aceptamos  $H_1$ . En consecuencia, se empleará la prueba de la U de Mann-Whitney para dictaminar si existen diferencias significativas entre REST y gRPC en lo que se refiere al tiempo de respuesta.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 124523$
  - $p = 0,917$

En esta ocasión, no se rechaza  $H_0$  en favor de  $H_1$  porque  $p > \alpha$ . De este modo, llegamos a la conclusión de que no existe diferencia significativa en el tiempo de respuesta de ambos ecosistemas en la obtención de volúmenes de datos pequeños al realizar el experimento con 500 muestras.

Con 300 muestras hemos determinado que el tiempo de respuesta de REST es significativamente menor que el de gRPC. Sin embargo, con 100 y 500 muestras hemos podido comprobar que no existe diferencia significativa entre el tiempo de respuesta de REST y gRPC. Por lo tanto, como en dos de los tres casos hemos visto que no existe

diferencia considerable, podemos llegar a la conclusión de que la diferencia entre el tiempo de respuesta de REST y gRPC en la obtención de volúmenes de datos pequeños no es significativa. En la Figura 24 se comparan mediante un diagrama de cajas los resultados de tiempo de respuesta en la obtención de volúmenes de datos pequeños con 100, 300 y 500 muestras entre REST y gRPC.



**Fig. 24.** Comparación de tiempos de respuesta en la obtención de volúmenes de datos pequeños entre REST y gRPC.

#### 5.1.1.2 Throughput

##### ▪ 100 muestras

En cuanto al *throughput*, el del ecosistema REST es un 0,4% mayor que el de gRPC, por lo que prácticamente no existe diferencia en la obtención de volúmenes de datos pequeños al realizar el experimento con 100 muestras.

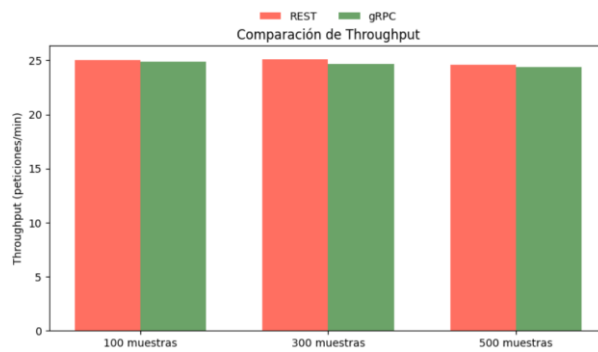
##### ▪ 300 muestras

Con 300 muestras, el *throughput* del ecosistema REST es un 1,62% mayor que el de gRPC, por lo que tampoco existe una diferencia importante.

##### ▪ 500 muestras

Al ejecutar el experimento con 500 muestras, el *throughput* del ecosistema REST es un 0,82% mayor que el de gRPC, así que también son prácticamente iguales.

En la Figura 25 se compara el *throughput* de REST y gRPC en la obtención de volúmenes pequeños de datos con 100, 300 y 500 muestras. Como se puede apreciar, no existen diferencias significativas en ningún caso.



**Fig. 25.** Comparación del *throughput* en la obtención de volúmenes de datos pequeños entre REST y gRPC.

#### 5.1.1.3 Consumo energético

- 100 muestras

En la obtención de volúmenes de datos de tamaño pequeño, al realizar el experimento con 100 muestras, el consumo de energía del entorno REST es un 3,91% superior al de gRPC.

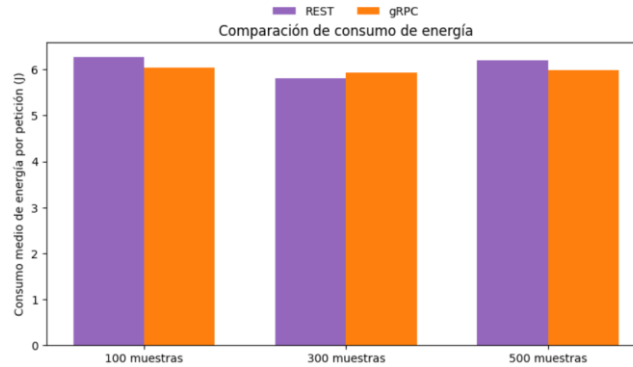
- 300 muestras

En esta ocasión, el consumo de energía del entorno gRPC es un 2,17% mayor que el de REST, por lo que no supone un gasto energético considerablemente superior.

- 500 muestras

Tras ejecutar el experimento con 500 muestras, observamos que el consumo de energía del entorno REST supera en un 3,44% al de gRPC.

Como podemos ver en la Figura 26, independientemente del número de muestras que se utilicen en el experimento, en la obtención de volúmenes de datos pequeños no hay prácticamente diferencia en el consumo energético de ambos entornos.



**Fig. 26.** Comparación del consumo energético en la obtención de volúmenes de datos pequeños entre REST y gRPC.

### 5.1.2 Experimentos de obtención de volúmenes de datos estándar

#### 5.1.2.1 Tiempo de respuesta

##### ▪ 100 muestras

Para comenzar, mediante Shapiro-Wilk, se comprobará si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,458$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,596$
  - $p_{gRPC} \approx 0$

Como  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , descartamos  $H_0$  y aceptamos  $H_1$ . Por lo tanto, llevaremos a cabo la prueba de la U de Mann-Whitney con el objetivo de ver si existen diferencias significativas entre REST y gRPC respecto al tiempo de respuesta.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 3080$
  - $p \approx 0$

Se rechaza  $H_0$  y se acepta de  $H_1$ , ya que  $p < \alpha$ , concluyendo que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la obtención de volúmenes de datos de tamaño estándar cuando en el experimento se usan 100 muestras.

▪ 300 muestras

Primeramente, utilizando Shapiro-Wilk, se comprobará si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,518$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,673$
  - $p_{gRPC} \approx 0$

$p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  y aceptamos  $H_1$ . Por consiguiente, llevaremos a cabo la prueba de la U de Mann-Whitney con el objetivo de ver si hay diferencias significativas entre REST y gRPC en el tiempo de respuesta.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 31391$
  - $p \approx 0$

Se descarta  $H_0$  en favor de de  $H_1$ , ya que  $p < \alpha$ , llegando a la conclusión de que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la obtención de volúmenes de datos de tamaño estándar cuando el experimento se lleva a cabo con 300 muestras.

▪ 500 muestras

Haciendo uso de Shapiro-Wilk, se comprobará si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.

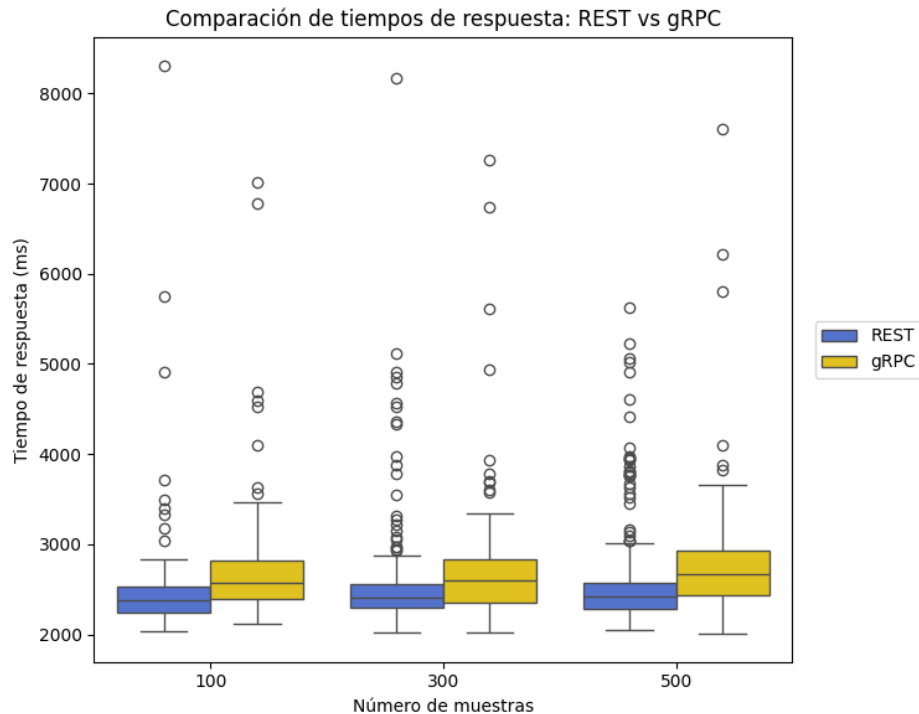
- $\alpha = 0,05$
- $W_{REST} = 0,631$
- $p_{REST} \approx 0$
- $W_{gRPC} = 0,775$
- $p_{gRPC} \approx 0$

Esta vez ha resultado  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , así que descartamos  $H_0$  y aceptamos  $H_1$ . De esta manera, ejecutaremos la prueba de la U de Mann-Whitney para ver si existen diferencias significativas entre REST y gRPC.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 77430$
  - $p \approx 0$

Se rechaza  $H_0$  y se acepta de  $H_1$ , ya que  $p < \alpha$ . Así pues, podemos garantizar que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la obtención de volúmenes de datos de tamaño estándar cuando el experimento se ejecuta con 500 muestras.

Con base en lo anterior, podemos concluir que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la obtención de volúmenes de datos estándar, aunque realizamos el experimento con 100, 300 o 500 muestras. En la Figura 27 se representa mediante un diagrama de cajas una comparativa de los resultados de tiempo de respuesta en la obtención de volúmenes de datos estándar con 100, 300 y 500 muestras entre REST y gRPC.



**Fig. 27.** Comparación de tiempos de respuesta en la obtención de volúmenes de datos estándar entre REST y gRPC.

#### 5.1.2.2 Throughput

##### ▪ 100 muestras

En lo que respecta al *throughput*, el de REST es un 9,22% mayor que el de gRPC en la obtención de volúmenes de datos estándar cuando en el experimento se emplean 100 muestras.

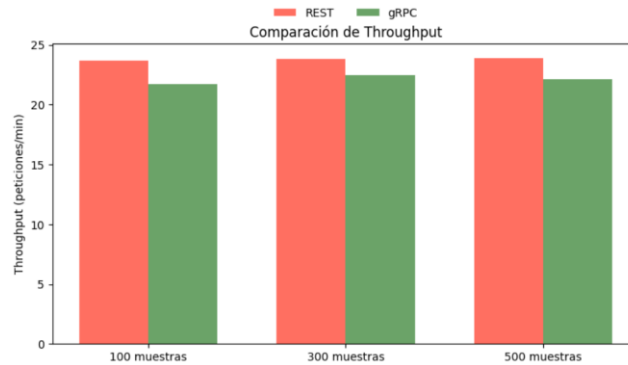
##### ▪ 300 muestras

En este caso, podemos apreciar que el *throughput* de los ecosistemas gRPC y REST son bastantes similares. Concretamente, el *throughput* de REST es un 5,78% mayor que el de gRPC.

##### ▪ 500 muestras

Al lanzar 500 peticiones en el experimento, determinamos que el *throughput* de REST es un 8,14% superior al de gRPC.

La Figura 28 representa, mediante un gráfico de barras, una comparativa del *throughput* de los entornos REST y gRPC en la obtención de volúmenes de datos estándar con 100, 300 y 500 muestras. Como se puede observar en la imagen, no existe una gran diferencia en el *throughput*, pero sí que podemos afirmar que el del entorno REST es superior al de gRPC, sin importar el tamaño de muestra empleado.



**Fig. 28.** Comparación del *throughput* en la obtención de volúmenes de datos estándar con entre REST y gRPC.

#### 5.1.2.3 Consumo energético

- 100 muestras

En cuanto al consumo energético en la obtención de volúmenes de datos de tamaño estándar con 100 muestras, gRPC ha obtenido un valor un 6,8% mayor que el de REST.

- 300 muestras

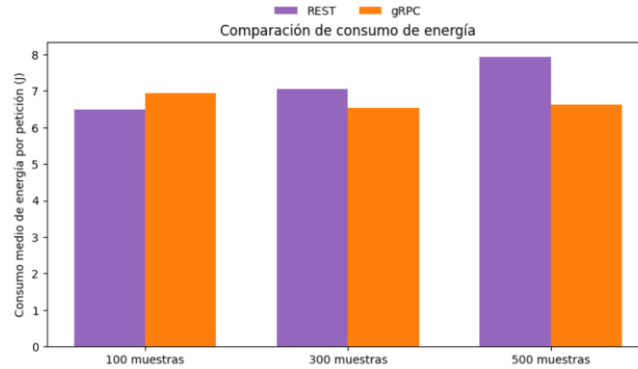
Al escoger 300 muestras, REST ha tenido un valor un 7,8% superior al de gRPC en la obtención de volúmenes de datos estándar.

- 500 muestras

Esta vez, ha resultado que el consumo energético de REST ha sido un 19,88% mayor que el de gRPC.

Como se puede apreciar en la Figura 29, con 100 muestras el consumo energético de gRPC ha sido mayor que el de REST, pero con 300 y 500 muestras gRPC ha sido más eficiente energéticamente que REST. En cualquier caso, la diferencia no es muy grande y los resultados no son del todo esclarecedores, por lo que podríamos decir que no existe una diferencia significativa en el consumo energético entre REST y gRPC en la obtención de volúmenes de datos de tamaño estándar.





**Fig. 29.** Comparación del consumo energético en la obtención de volúmenes de datos estándar entre REST y gRPC.

### 5.1.3 Experimentos de obtención de volúmenes de datos grandes

#### 5.1.3.1 Tiempo de respuesta

##### ▪ 100 muestras

Inicialmente, haremos uso del test de Shapiro-Wilk para verificar si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,675$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,638$
  - $p_{gRPC} \approx 0$

En este caso,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  en favor de  $H_1$ . Así pues, aplicaremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 5133,5$
  - $p = 0,745$

Tras ejecutar Mann-Whitney, vemos que no podemos rechazar  $H_0$ , ya que  $p > \alpha$ . De esta forma, podemos decir que no existe diferencia significativa entre el tiempo de respuesta de REST y gRPC en la obtención de volúmenes de datos de tamaño grande cuando el experimento se realiza con 100 muestras.

▪ 300 muestras

Utilizando el test de Shapiro-Wilk verificaremos si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,869$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,809$
  - $p_{gRPC} \approx 0$

En esta ocasión,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  en favor de  $H_1$ . En consecuencia, llevaremos a cabo el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 70299,5$
  - $p \approx 0$

Ahora vemos que hay que rechazar  $H_0$  y aceptar  $H_1$  porque  $p < \alpha$ . De ahí que podamos afirmar que el tiempo de respuesta de gRPC es significativamente menor que el de REST en la obtención de volúmenes de datos de tamaño grande al ejecutar el experimento con 300 muestras.

▪ 500 muestras

Mediante Shapiro-Wilk comprobaremos si ambos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$

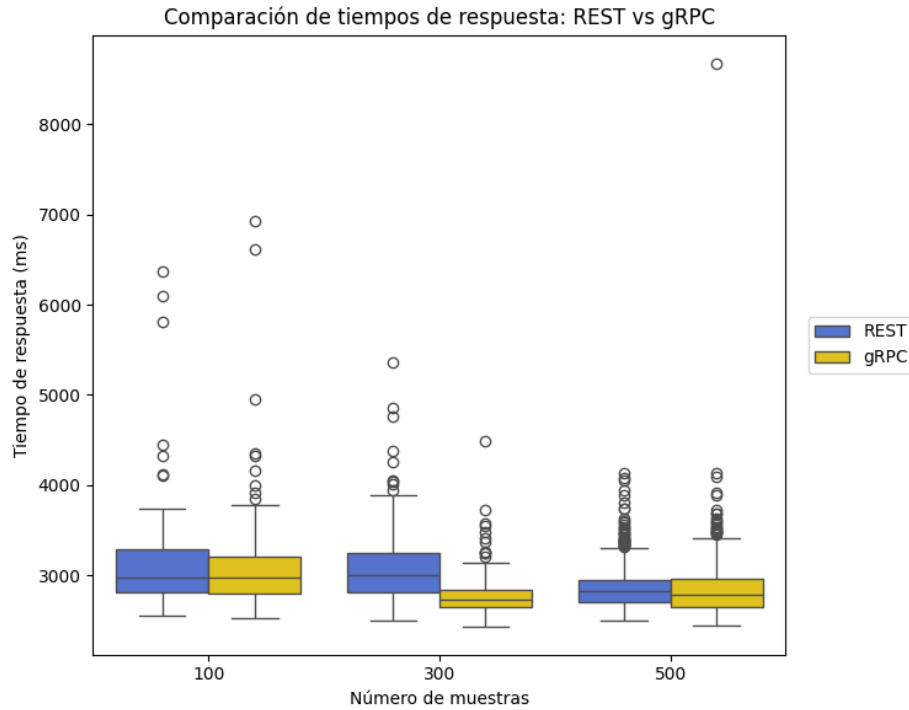
- $W_{REST} = 0,851$
- $p_{REST} \approx 0$
- $W_{gRPC} = 0,585$
- $p_{gRPC} \approx 0$

$p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , así que rechazamos  $H_0$  para aceptar  $H_1$ . De este modo, aplicaremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 139703,5$
  - $p = 0,001$

En este caso, observamos que debemos descartar  $H_0$  en favor de  $H_1$ , ya que  $p < \alpha$ . De esta forma, podemos decir que el tiempo de respuesta de gRPC es significativamente menor que el de REST en la obtención de volúmenes de datos de tamaño grande al llevar a cabo el experimento con 500 muestras.

Tras analizar, mediante estadística inferencial, los resultados obtenidos con 100, 300 y 500 muestras en la obtención de volúmenes de datos grandes, podemos concluir que el tiempo de respuesta de gRPC es significativamente menor que el de REST. Aunque con 100 muestras no se observan diferencias significativas, con 300 y 500 la diferencia sí resulta estadísticamente significativa. En el diagrama de cajas de la Figura 30 se puede apreciar esto visualmente.



**Fig. 30.** Comparación de tiempos de respuesta en la obtención de volúmenes de datos grandes entre REST y gRPC.

#### 5.1.3.2 Throughput

##### ▪ 100 muestras

En lo que corresponde al *throughput*, con 100 muestras, el de ambos ecosistemas es totalmente similar en la obtención de volúmenes de datos grandes.

##### ▪ 300 muestras

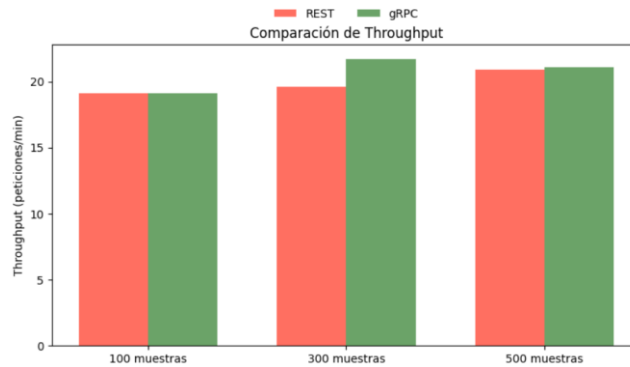
En este caso, al utilizar 300 muestras en el experimento, el *throughput* de gRPC es un 13,61% mayor que el de REST.

##### ▪ 500 muestras

Cuando en el experimento se escoge un tamaño de muestra de 500, el *throughput* de gRPC es un 0,96% superior al de REST.

Como se puede observar en el Figura 31, en la obtención de volúmenes de datos grandes, cuando utilizamos 100 muestras el *throughput* de ambos entornos es similar y cuando utilizamos 500 muestras también es prácticamente igual. Al emplear 300

muestras sí que el *throughput* de gRPC es mayor al de REST. En general, podríamos concluir que no existe grandes diferencias en el *throughput*, pero el de gRPC es ligeramente superior.



**Fig. 31.** Comparación del *throughput* en la obtención de volúmenes de datos grandes entre REST y gRPC.

#### 5.1.3.3 Consumo energético

- 100 muestras

En la obtención de volúmenes de datos de gran tamaño con 100 muestras, el consumo de energía es bastante parecido, siendo el de REST un 5,96% superior al de gRPC.

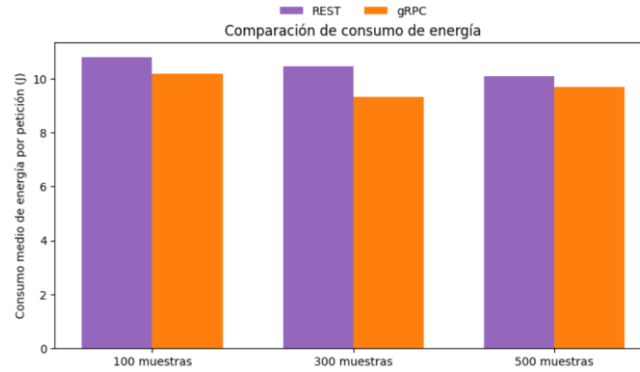
- 300 muestras

Al usar 300 muestras en la obtención de volúmenes de datos de gran tamaño, el consumo de energía de REST es un 12,22% mayor que el de gRPC.

- 500 muestras

En esta ocasión, el consumo de energía de REST es un 4,4% mayor que el de gRPC.

En resumen, tal y como representa la Figura 32, en la obtención de volúmenes de datos de gran tamaño, gRPC es más eficiente energéticamente que REST.



**Fig. 32.** Comparación del consumo energético en la obtención de volúmenes de datos grandes entre REST y gRPC.

## 5.2 Experimentos de creación de datos

### 5.2.1 Experimentos de creación de volúmenes de datos pequeños

#### 5.2.1.1 Tiempo de respuesta

##### ▪ 100 muestras

En primer lugar, se ejecutará la prueba de Shapiro-Wilk para determinar si los conjuntos de datos REST y gRPC siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,5$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,394$
  - $p_{gRPC} \approx 0$

$p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , así que rechazamos  $H_0$  en favor de  $H_1$ . A continuación, se llevará a cabo la prueba de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$

- $U = 621$
- $p \approx 0$

Esta vez, se rechaza  $H_0$  en favor de  $H_1$ , ya que  $p < \alpha$ . Finalmente, llegamos a la conclusión de que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos pequeños al utilizar 100 muestras.

▪ 300 muestras

Primeramente, se ejecutará la prueba de Shapiro-Wilk para determinar si ambos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,554$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,687$
  - $p_{gRPC} \approx 0$

En esta ocasión ha resultado  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , entonces descartamos  $H_0$  para aceptar  $H_1$ . Por consiguiente, se empleará la prueba de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 1873$
  - $p \approx 0$

Se rechaza  $H_0$  en favor de  $H_1$ , ya que  $p < \alpha$ . De esta manera, podemos garantizar que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos pequeños al usar 300 muestras.

▪ 500 muestras

Mediante el test de Shapiro-Wilk vamos a determinar si los conjuntos de datos REST y gRPC siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.

- $\alpha = 0,05$
- $W_{REST} = 0,863$
- $p_{REST} \approx 0$
- $W_{gRPC} = 0,888$
- $p_{gRPC} \approx 0$

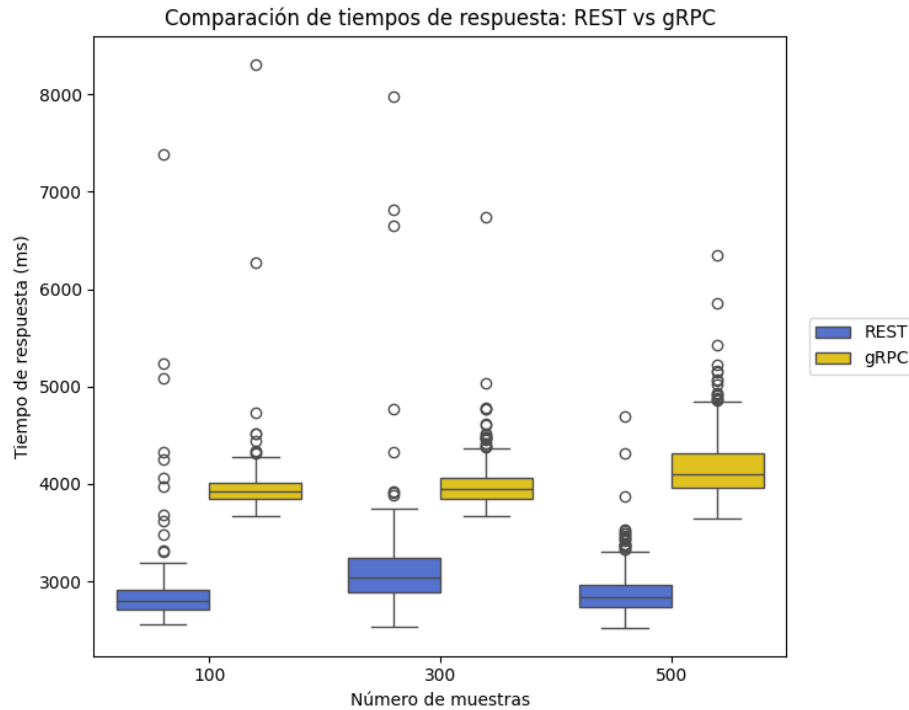
$p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que descartamos  $H_0$  en favor de  $H_1$ . En consecuencia, se llevará a cabo la prueba de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 904$
  - $p \approx 0$

En este caso, se rechaza  $H_0$  en favor de  $H_1$ , ya que  $p < \alpha$ . Por último, concluimos que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos pequeños cuando ejecutamos el experimento con 500 muestras.

Tras hacer un análisis estadístico inferencial de los resultados obtenidos con 100, 300 y 500 muestras, podemos afirmar que, independientemente del número de muestras, el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos pequeños. Esto se puede apreciar visualmente en la Figura 33.





**Fig. 33.** Comparación de tiempos de respuesta en la creación de volúmenes de datos pequeños entre REST y gRPC.

#### 5.2.1.2 Throughput

##### ▪ 100 muestras

En este caso, el *throughput* de REST es un 35,57% mayor que el de gRPC la creación de volúmenes de datos pequeños, por lo que la diferencia es considerable.

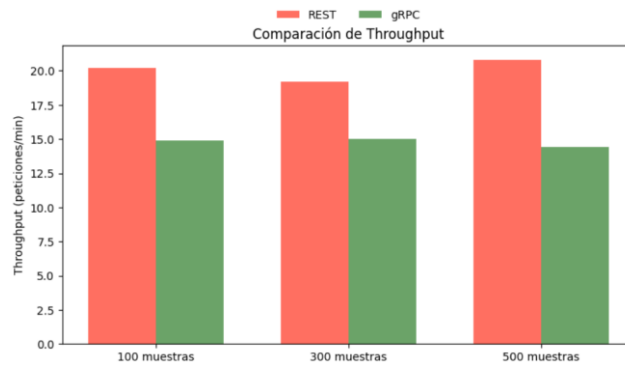
##### ▪ 300 muestras

Cuando para realizar el experimento se escoge un tamaño de muestra de 300, el *throughput* de REST es un 28% mayor que el gRPC.

##### ▪ 500 muestras

Cuando para llevar a cabo el experimento se emplea un tamaño de muestra de 500, el *throughput* de REST es un 44,44% superior al de gRPC.

De forma resumida, podemos concluir que, en la creación de volúmenes de datos pequeños, el *throughput* del entorno REST es considerablemente superior al del entorno gRPC. Esto se puede ver claramente en la Figura 34.



**Fig. 34.** Comparación del *throughput* en la creación de volúmenes de datos pequeños entre REST y gRPC.

#### 5.2.1.3 Consumo energético

##### ▪ 100 muestras

En la creación de volúmenes de datos de tamaño pequeño con 100 muestras, el consumo de energía del entorno REST es un 1,39% mayor que el de gRPC, por lo que la diferencia no es importante.

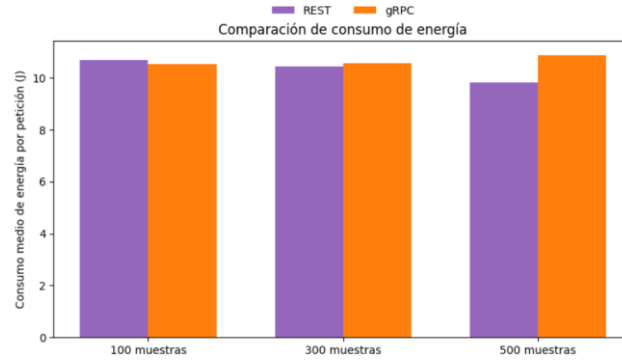
##### ▪ 300 muestras

Ahora, al usar 300 muestras en el experimento, el consumo energético de gRPC es un 1,24% superior al de REST.

##### ▪ 500 muestras

Esta vez el consumo energético del entorno gRPC ha resultado ser un 10,56% superior al de REST.

Con 100 y 300 muestras, el consumo energético en ambos entornos resulta prácticamente equivalente; solo al incrementarse a 500 muestras se aprecia una diferencia ligeramente mayor, aunque sin llegar a ser significativa. Finalmente, podemos concluir que no existe diferencia significativa entre el consumo energético de los entornos REST y gRPC en la creación de volúmenes de datos pequeños.



**Fig. 35.** Comparación del consumo energético en la creación de volúmenes de datos pequeños entre REST y gRPC.

### 5.2.2 Experimentos de creación de volúmenes de datos estándar

#### 5.2.2.1 Tiempo de respuesta

##### ▪ 100 muestras

Primeramente, mediante Shapiro-Wilk se comprobará si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,779$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,485$
  - $p_{gRPC} \approx 0$

Como  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , rechazamos  $H_0$  y aceptamos  $H_1$ , ejecutando, posteriormente, la prueba de la U de Mann-Whitney,

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 146$
  - $p \approx 0$

Se descarta  $H_0$  en favor de  $H_1$  porque  $p < \alpha$ . Así pues, llegamos a la conclusión de que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos de tamaño estándar.

▪ 300 muestras

Haciendo uso del test de Shapiro-Wilk verificaremos si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,588$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,579$
  - $p_{gRPC} \approx 0$

En este caso,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que descartamos  $H_0$  para aceptar  $H_1$ . Así pues, aplicaremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 5073$
  - $p \approx 0$

Tras ejecutar Mann-Whitney, vemos que debemos descartar  $H_0$  y aceptar  $H_1$ , ya que  $p < \alpha$ . De este modo, podemos decir que el tiempo de respuesta de REST es significativamente inferior al de gRPC en la creación de volúmenes de datos de tamaño estándar al realizar el experimento con 300 muestras.

▪ 500 muestras

Mediante Shapiro-Wilk comprobaremos si ambos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,865$

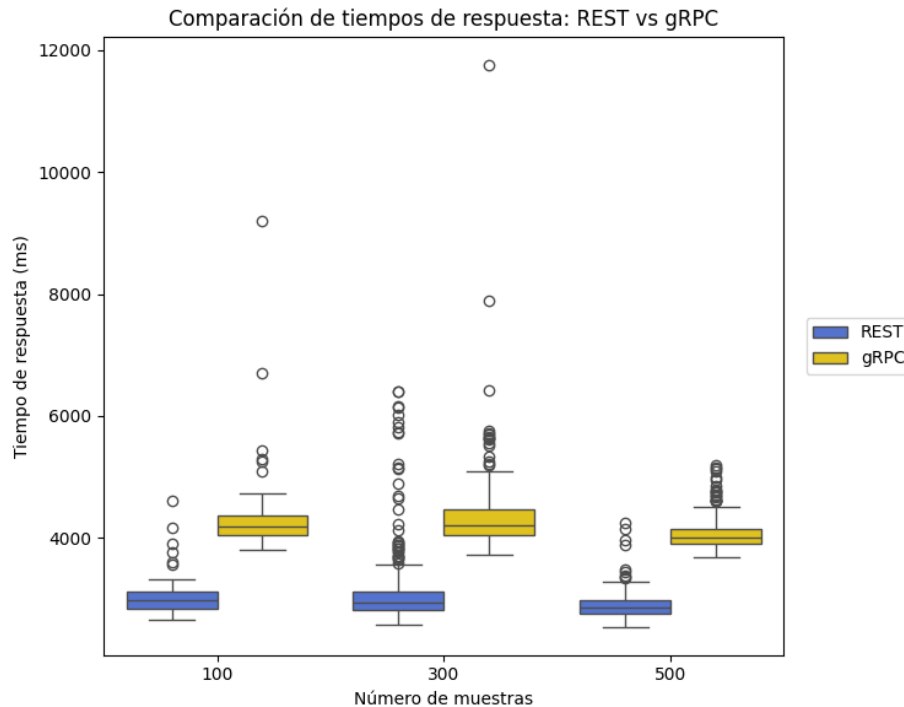
- $p_{REST} \approx 0$
- $W_{gRPC} = 0,869$
- $p_{gRPC} \approx 0$

$p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  en favor de  $H_1$ . Así pues, emplearemos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 1113,5$
  - $p \approx 0$

Se descarta  $H_0$  y se acepta  $H_1$ , porque  $p < \alpha$ . De esta forma, podemos afirmar que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos de tamaño estándar cuando el experimento se ejecuta con 500 muestras.

En vista de lo anterior, podemos concluir que el tiempo de respuesta de REST es significativamente menor que el de gRPC en la creación de volúmenes de datos estándar, aunque realizamos el experimento con 100, 300 o 500 muestras, tal y como se puede ver en la Figura 36.



**Fig. 36.** Comparación de tiempos de respuesta en la creación de volúmenes de datos estándar entre REST y gRPC.

#### 5.2.2.2 Throughput

- 100 muestras

El *throughput* de REST es un 42,45% superior al de gRPC en la creación de volúmenes de datos estándar cuando se usan 100 muestras.

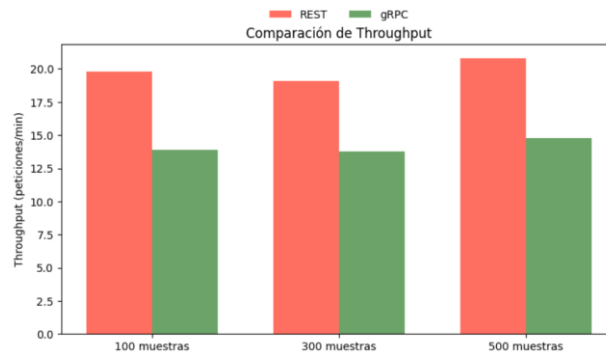
- 300 muestras

El *throughput* de REST es un 38,41% mayor que el de gRPC en la creación de volúmenes de datos estándar cuando se emplean 300 muestras en el experimento.

- 500 muestras

En este caso, el *throughput* de REST es un 40,54% mayor que el de gRPC.

En resumen, podemos afirmar que, en la creación de volúmenes de datos de tamaño estándar, el *throughput* del ecosistema REST es considerablemente superior al de gRPC, tal y como se puede apreciar en la Figura 37.



**Fig. 37.** Comparación del *throughput* en la creación de volúmenes de datos estándar entre REST y gRPC.

#### 5.2.2.3 Consumo energético

- 100 muestras

En la creación de volúmenes de datos estándar con 100 muestras, el consumo de energía entre ambos entornos es prácticamente igual. Específicamente, el consumo energético de REST es un 2,11% mayor que el de gRPC.

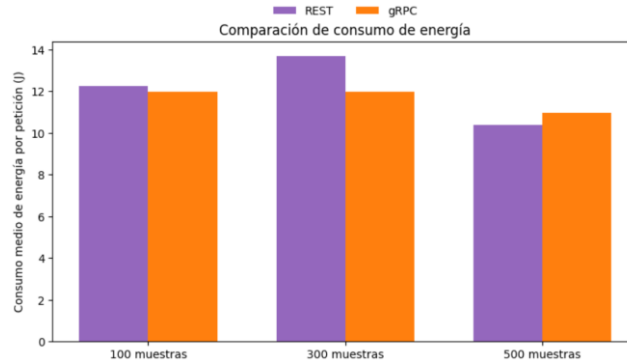
- 300 muestras

En este caso, al lanzar 300 peticiones al servidor, el consumo de energía de REST es un 14,24% superior al de gRPC.

- 500 muestras

Esta vez el consumo energético de gRPC ha resultado ser un 5,5% mayor que el de REST.

En lo relativo al consumo energético, se observa un comportamiento análogo al identificado en la generación de volúmenes de datos pequeños: según el tamaño de la muestra, gRPC resulta más eficiente energéticamente y, en otros casos, ocurre lo contrario. No obstante, las diferencias registradas nunca alcanzan valores considerables. De esta manera, podemos concluir que no existe diferencia significativa entre el consumo energético de los entornos REST y gRPC en la creación de volúmenes de datos tamaño estándar. La Figura 38 representa una comparativa del consumo energético en la creación de volúmenes de datos estándar entre ambos ecosistemas con 100, 300 y 500 muestras.



**Fig. 38.** Comparación del consumo energético en la creación de volúmenes de datos estándar entre REST y gRPC.

### 5.2.3 Experimentos de creación de volúmenes de datos grandes

#### 5.2.3.1 Tiempo de respuesta

##### ▪ 100 muestras

En primer lugar, ejecutaremos el test de Shapiro-Wilk.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,809$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,638$
  - $p_{gRPC} \approx 0$

En esta ocasión,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que rechazamos  $H_0$  en favor de  $H_1$ . En consecuencia, aplicaremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 7132,5$
  - $p \approx 0$



Como  $p < \alpha$ , debemos rechazar  $H_0$  y aceptar  $H_1$ . De este modo, llegamos a la conclusión de que el tiempo de respuesta de gRPC es significativamente menor que el de REST en la creación de volúmenes de datos de tamaño grande al emplear 100 muestras.

▪ 300 muestras

Inicialmente, haremos uso del test de Shapiro-Wilk para verificar si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,949$
  - $p_{REST} \approx 0$
  - $W_{gRPC} = 0,826$
  - $p_{gRPC} \approx 0$

En este caso,  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que descartamos  $H_0$  para aceptar  $H_1$ . Así pues, aplicaremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 79886$
  - $p \approx 0$

Tras ejecutar Mann-Whitney, vemos que debemos rechazar  $H_0$  y aceptar  $H_1$ , ya que  $p < \alpha$ . Finalmente, podemos garantizar que el tiempo de respuesta de gRPC es significativamente menor que el de REST en la creación de volúmenes de datos de tamaño grande al usar 300 muestras en el experimento.

▪ 500 muestras

Mediante el test de Shapiro-Wilk comprobaremos si los dos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{REST} = 0,617$

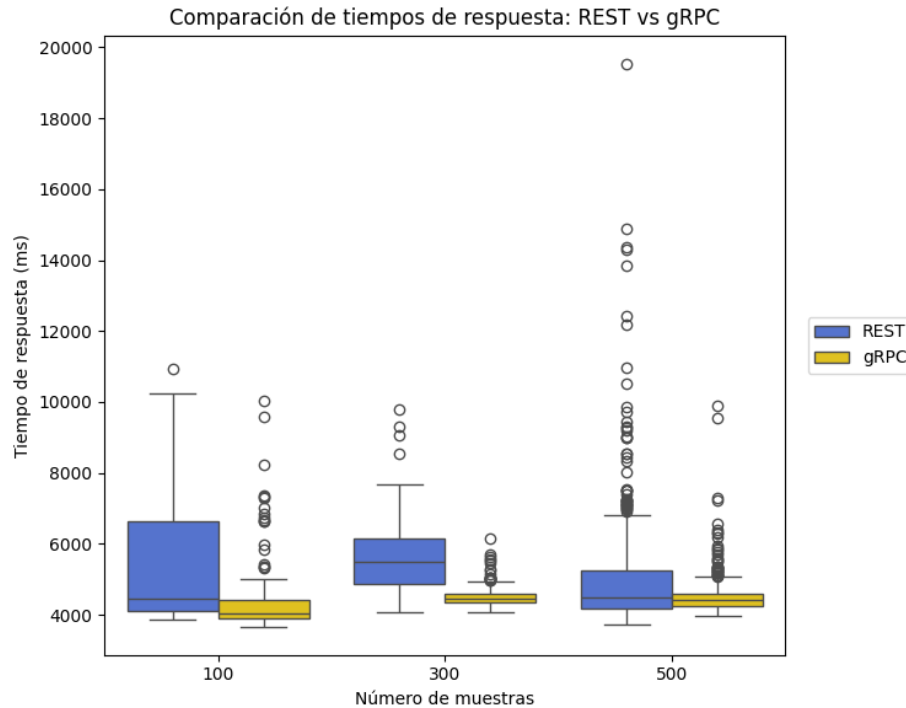
- $p_{REST} \approx 0$
- $W_{gRPC} = 0,585$
- $p_{gRPC} \approx 0$

Esta vez ha resultado  $p_{REST} < \alpha$  y  $p_{gRPC} < \alpha$ , por lo que descartamos  $H_0$  en favor de  $H_1$ . Por consiguiente, escogeremos el test de la U de Mann-Whitney.

- Mann-Whitney:
  - $H_0$ : no existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $H_1$ : existen diferencias significativas en el tiempo de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.
  - $\alpha = 0,05$
  - $U = 134548$
  - $p = 0,037$

Se descarta  $H_0$  y se acepta  $H_1$  porque  $p < \alpha$ . Por último, concluimos que el tiempo de respuesta de gRPC es significativamente menor que el de REST en la creación de volúmenes de datos de tamaño grande al utilizar 500 muestras.

Tras aplicar estadística inferencial a los resultados obtenidos con 100, 300 y 500 muestras durante la generación de grandes volúmenes de datos, se puede concluir que el tiempo de respuesta de gRPC es significativamente menor que el de REST, independientemente del número de muestras. Esto se observa de manera clara en la Figura 39.



**Fig. 39.** Comparación de tiempos de respuesta en la creación de volúmenes de datos grandes entre REST y gRPC.

#### 5.2.3.2 Throughput

##### ▪ 100 muestras

En lo que corresponde al *throughput*, al emplear 100 muestras en el experimento, el del ecosistema gRPC es un 18,02% superior al de REST en la creación de volúmenes de datos grandes.

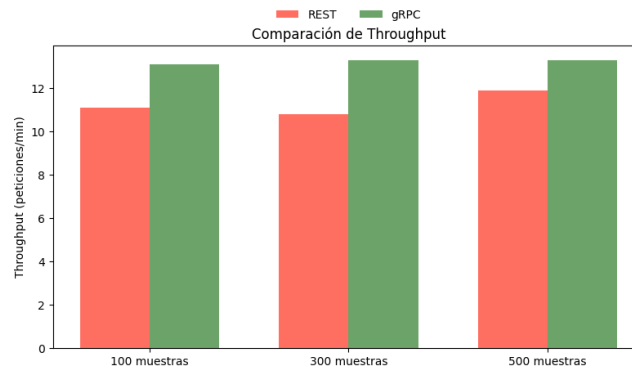
##### ▪ 300 muestras

Cuando se utilizan 300 muestras en el experimento, el *throughput* del ecosistema gRPC es un 23,81% mayor que el de REST.

##### ▪ 500 muestras

En esta ocasión, el *throughput* del entorno gRPC resulta un 11,76% superior al de REST al lanzar 500 peticiones al servidor.

Así pues, podemos garantizar que el *throughput* de gRPC es mayor que el de REST en la creación de volúmenes de datos de tamaño grande, tal y como se puede observar en la Figura 40.



**Fig. 40.** Comparación del *throughput* en la creación de volúmenes de datos grandes entre REST y gRPC.

#### 5.2.3.3 Consumo energético

- 100 muestras

En la creación de volúmenes de datos de gran tamaño, el consumo de energía de REST es un 69,12% superior al de gRPC cuando se lanzan 100 peticiones al servidor.

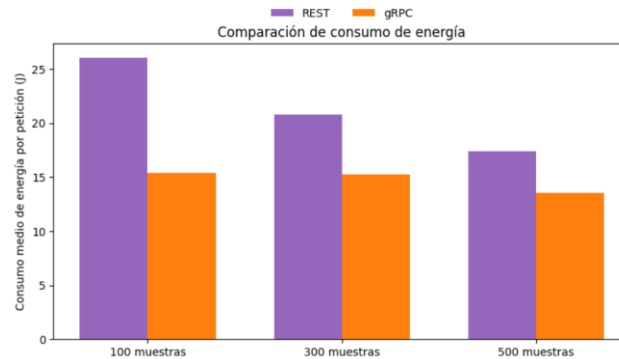
- 300 muestras

Cuando se escoge un tamaño de muestra de 300, el consumo de energía de REST es un 36,52% mayor que el de gRPC.

- 500 muestras

Con 500 muestras, el consumo de energía de REST es un 28,69% superior al de gRPC en la creación de volúmenes de datos de gran tamaño.

Como se puede ver en la Figura 41, el ecosistema gRPC es significativamente más eficiente energéticamente que el ecosistema REST en la creación de volúmenes de datos de tamaño grande.



**Fig. 41.** Comparación del consumo energético en la creación de volúmenes de datos grandes entre REST y gRPC.

### 5.3 Correlación entre el tiempo de respuesta y el consumo energético

En primer lugar, se han formado dos conjuntos de datos para evaluar si existe relación entre el tiempo de respuesta y el consumo de energía en la comunicación entre microservicios. En el primer conjunto de datos se han agrupado todos los resultados de tiempo medio de respuesta obtenidos tanto en los experimentos de obtención como de creación de datos. En el segundo conjunto de datos se han almacenado todos los valores de consumo energético medio por petición obtenidos en todos los experimentos realizados. Así pues, los dos conjuntos de datos obtenidos han sido los siguientes:

- Tiempo medio de respuesta = [2397; 2409; 2388; 2431; 2439; 2457; 2534; 2756; 2524; 2667; 2511; 2708; 3146; 3130; 3057; 2766; 2867; 2844; 2975; 4028; 3119; 3992; 2877; 4171; 3024; 4302; 3138; 4342; 2888; 4052; 5421; 4564; 5593; 4501; 5020; 4515]
- Consumo medio de energía = [6,273; 6,037; 5,801; 5,927; 6,192; 5,986; 6,503; 6,945; 7,059; 6,548; 7,935; 6,619; 10,793; 10,186; 10,451; 9,313; 10,101; 9,675; 10,676; 10,53; 10,439; 10,568; 9,83; 10,868; 12,241; 11,988; 13,691; 11,984; 10,397; 10,969; 26,044; 15,4; 20,783; 15,223; 17,424; 13,54]

A continuación, ejecutaremos la prueba de Shapiro-Wilk para determinar si ambos conjuntos de datos siguen una distribución normal.

- Shapiro-Wilk:
  - $H_0$ : ambos conjuntos siguen una distribución normal.
  - $H_1$ : al menos uno de los dos conjuntos no sigue una distribución normal.
  - $\alpha = 0,05$
  - $W_{\text{Tiempo de respuesta}} = 0,858$
  - $p_{\text{Tiempo de respuesta}} = 0,0003$

- $W_{\text{Consumo de energía}} = 0,849$
- $p_{\text{Consumo de energía}} = 0,0002$

En este caso,  $p_{\text{Tiempo de respuesta}} < \alpha$  y  $p_{\text{Consumo de energía}} < \alpha$ , por lo que rechazamos  $H_0$  en favor de  $H_1$ . Así pues, llevaremos a cabo la prueba del coeficiente de correlación de Spearman para determinar si existe relación estadísticamente significativa entre los dos conjuntos de datos.

- Spearman:
  - $H_0$ : no existe relación significativa entre el tiempo de respuesta y el consumo de energía en la comunicación entre microservicios.
  - $H_1$ : existe relación significativa entre el tiempo de respuesta y el consumo de energía en la comunicación entre microservicios.
  - $\alpha = 0,05$
  - $\rho = 0,953$
  - $p \approx 0$

Como  $p < \alpha$ , debemos rechazar  $H_0$  y aceptar  $H_1$ . Finalmente, podemos concluir que existe relación estadísticamente significativa entre el tiempo de respuesta y el consumo energético en la comunicación entre microservicios.

## 6 Evaluación de los resultados

### 6.1 Respuestas a las preguntas de investigación

A continuación, basándonos en el análisis estadístico realizada previamente, responderemos a las preguntas de investigación planteadas.

- RQ1: ¿Qué mecanismo de comunicación tiene mejor rendimiento en la obtención de volúmenes de datos de diferente tamaño?
  - En primer lugar, en la obtención de volúmenes de datos pequeños, hemos concluido que no existe diferencia estadísticamente significativa entre REST y gRPC; en cuanto al *throughput*, las diferencias con cualquier tamaño de muestra también han sido insignificantes. Por otro lado, al trabajar con volúmenes de datos de tamaño estándar, REST ha logrado un tiempo de respuesta significativamente inferior al de gRPC, y su *throughput* ha sido mayor que el de gRPC con 100, 300 y 500 muestras. En contraste, en la obtención de volúmenes de datos grandes, hemos llegado a la conclusión de que gRPC ha presentado un tiempo de respuesta significativamente más bajo que REST; respecto al *throughput*, el de gRPC ha sido ligeramente superior. En síntesis, REST ofrece un mejor rendimiento en la obtención de volúmenes de datos de tamaño estándar, mientras que gRPC resulta más eficiente en la obtención de volúmenes de datos grandes. En la obtención de volúmenes de datos pequeños no hay diferencia significativa entre ambos entornos. La hipótesis inicial formulaba que gRPC tendría un rendimiento superior en la obtención de datos, independientemente del tamaño, especialmente en escenarios con grandes volúmenes de datos. A la luz de los resultados obtenidos en los distintos experimentos, se puede afirmar que dicha hipótesis se cumple parcialmente, ya que gRPC ha demostrado ser más eficiente en la obtención de volúmenes de datos grandes, pero no ha sido así con volúmenes de datos pequeños y estándar.
- RQ2: ¿Qué mecanismo de comunicación consume menos energía en la obtención de volúmenes de datos de diferente tamaño?
  - Primeramente, en la obtención de volúmenes de datos pequeños, hemos deducido que no existe diferencia significativa entre el consumo energético de REST y gRPC. Por otro lado, al trabajar con volúmenes de datos de tamaño estándar los resultados no han sido del todo esclarecedores, pero, en cualquier caso, las diferencias han sido ligeras. Finalmente, en el caso de volúmenes de datos grandes, gRPC ha consumido menos energía que REST, independientemente del tamaño de muestra, pero las diferencias tampoco han sido muy grandes. En conjunto, puede afirmarse que no existen diferencias relevantes en el consumo energético entre REST y gRPC al obtener datos; únicamente en volúmenes grandes gRPC resulta ligeramente más eficiente energéticamente que REST. En un primer momento, se formuló la hipótesis de que REST tendría un menor consumo energético que gRPC en todos los escenarios de obtención

de datos. A partir de los resultados obtenidos, podemos concluir que dicha hipótesis no se verifica en absoluto, ya que REST no ha demostrado tener mejor rendimiento energético en ningún caso.

- RQ3: ¿Qué mecanismo de comunicación ofrece mejor rendimiento en la creación de volúmenes de datos de diferente tamaño?
  - En primer lugar, en la creación de volúmenes de datos pequeños, REST ha conseguido mejor resultado que gRPC en el tiempo de respuesta, obteniendo una diferencia estadísticamente significativa; respecto al *throughput*, también ha sido ampliamente superior en REST. En segundo lugar, en la creación de volúmenes de datos de tamaño estándar, REST ha logrado un tiempo de respuesta significativamente menor que el de gRPC; el *throughput*, por su parte, también ha sido mayor en el ecosistema REST. Finalmente, en la creación de volúmenes de datos de tamaño grande, el tiempo de respuesta de gRPC ha sido significativamente menor que el de REST; en cuanto al *throughput*, el de gRPC ha sido mayor que el de REST, independientemente de si se lanzan 100, 300 o 500 peticiones al servidor. Así pues, REST ofrece mejor rendimiento en la creación de volúmenes de datos de tamaño pequeño y estándar, mientras que gRPC es más eficiente en la creación de volúmenes de datos grandes. La hipótesis formulada en primera instancia fue que REST ofrece mejor rendimiento en la creación de datos, especialmente para volúmenes de datos pequeños y estándar. Tras evaluar los resultados obtenidos en los diferentes experimentos, podemos afirmar que la hipótesis se cumple en gran medida, con la salvedad de que en la creación de volúmenes grandes de datos gRPC ofrece un mejor rendimiento.
- RQ4: ¿Qué mecanismo de comunicación consume menos energía en la creación de volúmenes de datos de diferente tamaño?
  - Por un lado, en la creación de volúmenes de datos pequeños, hemos determinado que no existe diferencia significativa entre el consumo energético de REST y gRPC. Por otro lado, en la creación de volúmenes de datos de tamaño estándar, tampoco existe una diferencia significativa entre ambos mecanismos de comunicación. Por último, en la creación de volúmenes de datos de tamaño grande, gRPC ha logrado mejores resultados que REST. En definitiva, la única diferencia significativa en el consumo energético se da en la generación de grandes volúmenes de datos, donde gRPC resulta más eficiente. Al inicio de este trabajo, se formuló la hipótesis de que REST consume menos energía que gRPC en la creación de volúmenes de datos de cualquier tamaño, pero ahora, en vista de los resultados obtenidos, se puede afirmar que la hipótesis era totalmente errónea.
- RQ5: ¿Existe relación entre el rendimiento y el consumo de energía en la comunicación entre microservicios?
  - Al inicio de este estudio, se formuló la hipótesis de que existe relación entre el rendimiento y el consumo de energía en la comunicación entre



microservicios. Tras realizar un análisis de los resultados mediante estadística inferencial, podemos garantizar que dicha hipótesis es completamente correcta.

## 6.2 Amenazas a la validez

En este estudio se identifican diversas amenazas a la validez que podrían afectar la interpretación y generalización de los resultados.

### 6.2.1 Validez interna

En relación con la validez interna, el uso de una máquina compartida con otros procesos no controlados introduce una fuente potencial de variabilidad en los resultados de rendimiento y consumo energético, lo cual podría comprometer la atribución causal entre los entornos evaluados y los parámetros medidos.

### 6.2.2 Validez externa

En cuanto a la validez externa, una limitación relevante es que cada entorno evaluado (REST y gRPC) solo incluye dos microservicios, lo cual difiere de escenarios reales donde suele haber múltiples servicios interconectados. Esta simplificación puede limitar la extrapolación de los resultados a sistemas de mayor complejidad. Asimismo, el hecho de que los microservicios se ejecuten en una única máquina local, en lugar de estar distribuidos en distintos servidores como ocurre comúnmente en entornos productivos, puede influir en el rendimiento observado, afectando también la validez externa del estudio. Por último, en todos los escenarios definidos en el apartado 4.2.2 se ha seleccionado un *payload* de tamaño fijo, lo cual difiere de las aplicaciones reales, donde el tamaño del *payload* suele variar dentro de un rango acotado; esto también puede limitar la extrapolación de los resultados a casos reales.

### 6.2.3 Validez de constructo

En lo que respecta a la validez de constructo, la medición del consumo energético a nivel de CPU de la máquina completa, en lugar de a nivel de contenedor, puede no reflejar fielmente el constructo de “consumo energético en la comunicación entre microservicios en cada entorno”. Esta discrepancia afecta la coherencia entre la variable observada y el fenómeno que se desea evaluar, ya que en los núcleos de la CPU pueden estar ejecutándose otros procesos no controlados que influyen en la métrica registrada. Cabe destacar que lo ideal hubiese sido medir de forma aislada la energía consumida por los contenedores Docker correspondientes a cada entorno. No obstante, debido a limitaciones técnicas (concretamente, la imposibilidad de utilizar un sistema operativo Linux en la máquina experimental), no ha sido posible emplear herramientas especializadas, como por ejemplo DEEP-mon [53], desarrollada por NECSTLab en el Politecnico di Milano, que permite monitorizar con precisión el consumo energético a nivel

de contenedor. A pesar de esta limitación, dado que todos los experimentos se han ejecutado en la misma máquina y bajo las mismas condiciones (intentando que los procesos en ejecución sean los mismos en todo momento), se podría considerar que los resultados obtenidos son relativamente fiables para realizar una comparación entre los mecanismos de comunicación REST y gRPC en cuanto a su eficiencia energética.

#### **6.2.4 Validez estadística-conclusiva**

El uso de una máquina compartida con otros procesos no controlados también incide en la validez estadística-conclusiva, dado que la interferencia de procesos ajenos al experimento puede introducir ruido en los datos y reducir la precisión de las conclusiones inferidas.

## 7 Conclusiones

El presente trabajo ha tenido como propósito analizar comparativamente los mecanismos de comunicación REST y gRPC en términos de rendimiento y consumo energético, en escenarios de obtención y creación de volúmenes de datos de diferente tamaño. Para ello, se formularon diversas hipótesis iniciales, que posteriormente fueron contrastadas con los resultados experimentales.

En primer lugar, respecto al rendimiento en la obtención de datos, se ha constatado que no existen diferencias estadísticamente significativas en volúmenes pequeños, mientras que REST ha mostrado un mejor desempeño con volúmenes de datos estándar, tanto en tiempo de respuesta como en *throughput*. Sin embargo, cuando se trata de volúmenes grandes de datos, gRPC ha demostrado ser más eficiente en ambas métricas. Por tanto, la hipótesis que planteaba una ventaja general de gRPC en la obtención de datos únicamente se cumple parcialmente, pues esta ventaja solo se ha evidenciado en escenarios con cargas elevadas.

En cuanto al consumo energético en la obtención de datos, los resultados han mostrado que las diferencias entre ambos mecanismos son poco relevantes en volúmenes pequeños y estándar, mientras que en volúmenes grandes gRPC ha sido ligeramente más eficiente. En este sentido, la hipótesis inicial, que sostenía una ventaja energética de REST en todos los escenarios, no se ha verificado, ya que REST no ha presentado un consumo inferior en ninguna de las pruebas.

Por lo que respecta al rendimiento en la creación de datos, se ha observado que REST ofrece mejores resultados con volúmenes pequeños y estándar, tanto en tiempo de respuesta como en *throughput*. No obstante, para la creación de grandes volúmenes de datos, gRPC ha sido más eficiente, mostrando tiempos de respuesta significativamente menores y un *throughput* superior. De este modo, la hipótesis inicial (según la cual REST tendría un mejor rendimiento, especialmente en volúmenes pequeños y estándar) puede considerarse correcta en gran parte, aunque con el matiz de que en escenarios de gran carga gRPC supera a REST.

En lo relativo al consumo energético en la creación de datos, se ha determinado que no existen diferencias significativas en volúmenes pequeños y estándar, mientras que en volúmenes grandes gRPC ha mostrado una clara ventaja. En consecuencia, la hipótesis que planteaba un menor consumo energético de REST en todos los escenarios ha quedado refutada, siendo gRPC el mecanismo que ha demostrado una mejor eficiencia energética en los casos de mayor demanda.

Finalmente, respecto a la posible relación entre rendimiento y consumo energético en la comunicación entre microservicios, los análisis estadísticos realizados han confirmado de forma concluyente la hipótesis inicial: existe una correlación clara entre ambas variables. Así, los entornos en los que un mecanismo presenta mejor rendimiento tienden también a ser más eficientes energéticamente.

En síntesis, puede afirmarse que REST y gRPC no presentan diferencias significativas en escenarios con volúmenes de datos pequeños, mientras que REST resulta más adecuado para volúmenes estándar y gRPC se muestra más eficiente en escenarios de gran carga, tanto en términos de rendimiento como de consumo energético. Estas conclusiones ponen de manifiesto que la elección de un mecanismo de comunicación debe estar condicionada por el tipo de carga de trabajo y el contexto específico de la aplicación.

## 8 Líneas de investigación futuras

Este estudio comparativo entre REST y gRPC en términos de rendimiento y consumo energético ha permitido obtener conclusiones relevantes, pero también ha abierto varias vías para investigaciones futuras que podrían enriquecer y ampliar los resultados presentados.

En primer lugar, una posible línea de trabajo consiste en replicar el estudio utilizando un entorno experimental más aislado y controlado. En esta investigación, ambos ecosistemas (REST y gRPC) se han implementado con dos microservicios cada uno, ejecutándose en contenedores Docker sobre una máquina local. Para eliminar posibles interferencias derivadas de procesos no controlados en el entorno local, sería conveniente desplegar los ecosistemas en servidores dedicados. De forma más avanzada, y para reflejar con mayor fidelidad escenarios reales, podría evaluarse el desempeño y consumo energético desplegando cada microservicio en servidores distintos. Esta configuración permitiría analizar la comunicación entre microservicios cuando se realiza a través de diferentes servidores, tal y como ocurre normalmente en entornos de producción.

Una segunda línea de investigación futura consiste en ampliar el número de microservicios en cada ecosistema, lo que permitiría simular escenarios más complejos y realistas, y estudiar cómo varía el comportamiento en función del tamaño y la complejidad del sistema. Esta línea es compatible con la anterior y ambas podrían combinarse para profundizar en el impacto del despliegue y la escalabilidad.

Un tercer aspecto a mejorar es la medición del consumo energético. Debido a limitaciones técnicas, los microservicios se ejecutaron en un sistema operativo Windows, utilizando Intel Power Gadget, que únicamente permite medir el consumo energético a nivel de CPU y no a nivel de contenedor. En futuras investigaciones, sería recomendable utilizar servidores con Linux, ya que para este sistema operativo existen herramientas que permiten medir el consumo energético a nivel de contenedor, lo que permitiría obtener mediciones más precisas y detalladas. Esta línea es compatible con las dos anteriores, por lo que sería muy interesante combinarlas en un estudio futuro.

Adicionalmente, otra línea de investigación relevante es la incorporación de nuevas variables dependientes en el análisis. Este estudio se ha centrado en el tiempo de respuesta, el *throughput* y el consumo energético, pero se podrían añadir otras métricas como el consumo de CPU o memoria, con el fin de obtener una visión más completa.

Por otro lado, aunque la comparación se ha realizado entre REST y gRPC, el análisis podría extenderse para incluir otros estilos arquitectónicos de APIs, como GraphQL o SOAP. En particular, sería enriquecedor añadir GraphQL en el estudio, ya que es uno de los estilos arquitectónicos más utilizados en la actualidad.

Finalmente, en esta investigación los escenarios de prueba se definieron en función del tamaño del *payload*. Sería interesante explorar escenarios alternativos basados en

características diferentes, como el uso de datos anidados frente a datos planos, o patrones específicos de carga, para evaluar cómo estas variables afectan el rendimiento y consumo energético en cada ecosistema.

En definitiva, estas líneas de investigación futuras ofrecen un amplio campo de oportunidades para seguir profundizando en el estudio comparativo de mecanismos de comunicación en arquitectura de microservicios, contribuyendo a la mejora y optimización de sistemas distribuidos.

## Referencias

- [1] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, *Microservices: yesterday, today, and tomorrow*. Springer International Publishing, 2017, pp. 195–216.
- [2] C. O’Hanlon, “A Conversation with Werner Vogels: Learning from the Amazon technology platform: Many think of Amazon as ‘that hugely successful online bookstore.’ You would expect Amazon CTO Werner Vogels to embrace this distinction, but in fact it causes him some concern.” *Queue*, vol. 4, pp. 14–22, 2006.
- [3] C. Surianarayanan, G. Ganapathy, and P. Raj, *Essentials of Microservices Architecture : Paradigms, Applications, and Techniques*. CRC Press LLC, 2019.
- [4] W3C, “SOAP Version 1.2 Part 1: Messaging Framework,” <https://www.w3.org/TR/2003/REC-soap12-part1-20030624/>.
- [5] RFC, “RFC 2616: Hypertext Transfer Protocol -- HTTP/1.1,” <https://www.rfc-editor.org/rfc/rfc2616.html>.
- [6] R. T. Fielding and R. N. Taylor, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, 2000.
- [7] R. K. Soni and N. Soni, *Spring Boot with React and AWS*. Apress, 2021.
- [8] S. Sharma, *Modern API Development with Spring and Spring Boot*. Packt Publishing, 2021.
- [9] M. Daz, “Entendiendo las APIs RESTful y el Protocolo HTTP,” <https://dev.to/mil-tondiazco/entendiendo-las-apis-restful-y-el-protocolo-http-44a6>.
- [10] RFC, “RFC 707: A High-Level Framework for Network-Based Resource Sharing,” <https://www.rfc-editor.org/rfc/rfc707.html>.
- [11] A. Giretti, *Beginning gRPC with ASP.NET Core 6*. Apress, 2022.
- [12] S. Buna, *GraphQL in Action*. Manning Publications Co., 2021.
- [13] M. Dulak and L. Dynowski, *Learning API Styles*. O’Reilly Media, Inc., 2025.
- [14] C. Freitag, M. Berners-Lee, K. Widdicks, B. Knowles, G. S. Blair, and A. Friday, “The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations,” *Patterns*, vol. 2, no. 9, 2021.

[15] Ministerio para la Transición Ecológica y el Reto Demográfico, “Objetivos de reducción de emisiones de gases de efecto invernadero,” <https://www.miteco.gob.es/es/cambio-climatico/temas/mitigacion-politicas-y-medidas/objetivos.html>.

[16] Postman, “2024 State of the API Report,” <https://www.postman.com/state-of-api/2024/>.

[17] Postman, “2023 State of the API Report,” <https://www.postman.com/state-of-api/2023/>.

[18] Postman, “2020 State of the API Report,” <https://www.postman.com/state-of-api/2020/>.

[19] Postman, “2022 State of the API Report,” <https://www.postman.com/state-of-api/2022/>.

[20] Y. Zhao, T. De Matteis, and J. Bogner, “How Does Microservice Granularity Impact Energy Consumption and Performance? A Controlled Experiment,” in *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*, 2025, pp. 84–95.

[21] L. O. Soares and R. Terra, “Go x Java e gRPC x REST: Um estudo empírico,” in *Anais do XXVIII Simpósio Brasileiro de Linguagens de Programação*. SBC, 2024, pp. 62–70.

[22] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, “Performance evaluation of microservices communication with REST, GraphQL, and gRPC,” *International Journal of Electronics and Telecommunication*, vol. 70, no. 2, pp. 429–436, 2024.

[23] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, “A Comparative Analysis of Communication Efficiency: REST vs. gRPC in Microservice-Based Ecosystems,” in *2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*, 2024, pp. 621–626.

[24] I. Olivos and M. Johansson, “Comparative Study of REST and gRPC for Microservices in Established Software Architectures,” 2023.

[25] Ł. Kamiński, M. Kozłowski, D. Sporysz, K. Wolska, P. Zaniewski, and R. Roszczyk, “Comparative review of selected Internet communication protocols,” *Foundations of Computing and Decision Sciences*, vol. 48, no. 1, pp. 39–56, 2022.

[26] M. Bolanowski, K. Żak, A. Paszkiewicz, M. Ganzha, M. Paprzycki, P. Sowiński, I. Lacalle, and C. E. Palau, “Efficiency of REST and gRPC realizing communication tasks in microservice-based ecosystems,” in *Proc. 21st Int. Conf. New Trends Intell. Softw. Methodol.*, vol. 355, 2022, pp. 97–108.



- [27] M. Śliwa and B. Pańczyk, “Performance comparison of programming interfaces on the example of REST API, GraphQL and gRPC,” *Journal of Computer Sciences Institute*, vol. 21, pp. 356–361, 2021.
- [28] C. L. Chamas, D. Cordeiro, and M. M. Eler, “Comparing REST, SOAP, Socket and gRPC in computation offloading of mobile applications: an energy cost analysis,” in *2017 IEEE 9th Latin-American Conference on Communications (LATINCOM)*. IEEE, 2017, pp. 1–6.
- [29] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
- [30] A. D. Singh, *Building and Delivering Microservices on AWS*, 1st ed. Packt Publishing, 2023.
- [31] V. Velepucha and P. Flores, “A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,” *IEEE Access*, vol. 11, 2023.
- [32] V. Bojinov, *RESTful Web API Design with Node.js*. Packt Publishing, 2016.
- [33] G. Gitonga, “Understanding Rest Architecture,” <https://www.linkedin.com/pulse/understanding-rest-architecture-gabriel-gitonga>.
- [34] H. Subramanian and P. Raj, *Hands-On RESTful API Design Patterns and Best Practices*. Packt Publishing, 2019.
- [35] gRPC.io, “gRPC - A high-performance, open-source universal RPC framework,” <https://grpc.io/>.
- [36] K. Indrasiri and D. Kuruppu, *gRPC: Up and Running*. O’Reilly Media, 2020.
- [37] M. Genero, J. A. Cruz-Lemus, and M. G. Piattini, *Métodos de investigación en ingeniería del software*. Ra-Ma, 2015.
- [38] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [39] Oracle, “Oracle Database gratuita para todos,” <https://www.oracle.com/es/database/technologies/appdev/xe.html/>.
- [40] G. Schenker, *Learn Docker - fundamentals of Docker 19.x : build, test, ship, and run containers with Docker and Kubernetes*. Packt Publishing, 2020.
- [41] J.-P. Gouigoux, *Docker: Primeros pasos y puesta en práctica de una arquitectura basada en micro-servicios*. Ediciones ENI, 2018.
- [42] Docker, “Docker - Develop faster. Run everywhere,” <https://www.docker.com/>.

[43] OpenAPI Initiative, “OpenAPI Specification v3.0.0,” <https://spec.openapis.org/oas/v3.0.0.html>.

[44] Protocol Buffers Documentation, “Language Guide (proto 3),” <https://protobuf.dev/programming-guides/proto3/>.

[45] JetBrains, “IntelliJ IDEA Ultimate vs IntelliJ IDEA Community Edition,” <https://www.jetbrains.com/products/compare/?product=idea&product=idea-ce>.

[46] Oracle, “Java SE 17 Archive Downloads (17.0.12 and earlier),” <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>.

[47] Spring, “Spring Initializr,” <https://start.spring.io/>.

[48] Maven, “Apache Maven Project,” <https://maven.apache.org/docs/3.8.7/release-notes.html>.

[49] GitHub, “Build and ship software on a single, collaborative platform,” <https://github.com/>.

[50] Apache JMeter, “Apache JMeter,” <https://jmeter.apache.org/>.

[51] Intel, “Using the Intel® Power Gadget 3.0 API on Windows\*,” <https://www.intel.com/content/www/us/en/developer/articles/training/using-the-intel-power-gadget-30-api-on-windows.html>.

[52] C. Rodriguez, M. Baez, F. Daniel, F. Casati, J. Trabucco, L. Canali, and G. Percannella, “REST APIs: A Large-Scale Analysis of Compliance with Principles and Best Practices,” in *Web Engineering*, 2016, pp. 21–39.

[53] R. Brondolin, T. Sardelli, and M. D. Santambrogio, “DEEP-Mon: Dynamic and Energy Efficient Power Monitoring for Container-Based Infrastructures,” in *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2018, pp. 676–684.